دوره جامع مهندسی نرم‌افزار ایجنتیک (Agentic AI) ۲۰۲۶

راهنمای جامع وایب کدینگ با کلود کد (Claude Code)

آموزش صفر تا صد، مفهومی و عملی از مبانی ترمینال تا استقرار پروژه‌های مدرن بر مبنای سرفصل‌های رسمی Vibe Code School

۵ فصل تخصصی 🚀 | ۲۸ درس پروژه محور 📚 | چیت‌شیت جامع ۱۳ گانه 🛠️ | واژه‌نامه ۸۳ اصطلاح هوش مصنوعی 📖 | آزمون‌های تستی با تشریح پاسخ 💡

مرجع و منبع اصلی:	**Vibe Code School (Dinesh Puppala) & Anthropic**
ترجمه، تبیین و تنظیم تخصصی:	**مهدی حیدرپور (MahdiDev)**
تایپوگرافی و قلم:	**فونت وزیرمتن (Vazirmatn) – کاملاً راست‌به‌چپ (RTL)**
تاریخ انتشار:	**آبان ۱۴۰۳ (مطابق با نسخه ۲۰۲۶)**

سخن آغازین: مانیفست وایب کدینگ و عصر ایجنت‌های هوشمند

در سال‌های اخیر، مفهوم برنامه‌نویسی و مهندسی نرم‌افزار با بزرگ‌ترین زلزله پارادایمی تاریخ خود مواجه شده است. اصطلاح **وایب کدینگ (Vibe Coding)** که نخستین بار توسط *آندری کارپاتی (Andrej Karpathy)*، از بنیان‌گذاران پیشین OpenAI و مدیر اسبق هوش مصنوعی تسلا مطرح گردید، به یک تغییر رویکرد شگرف اشاره دارد: **پایان عصر تایپ کاراکتر به کاراکتر سینتکس و آغاز عصر هدایت نتایج نهایی به وسیله ایجنت‌ها.**

> **تعریف رسمی وایب کدینگ:** «ساخت نرم‌افزارهای کامل از طریق توصیف آنچه می‌خواهید به یک ایجنت خودمختار، ارزیابی هوشمندانه نتایج و هدایت مسیر، به جای درگیر شدن با جزئیات خسته‌کننده خطوط کد.»

تفاوت بنیادین ابزارهای تکمیل خودکار با Claude Code

ابزارهای نسل پیشین (مانند افزونه‌های Autocomplete قدیمی در ادیتورها) مانند یک دستیار منفعل عمل می‌کردند که صرفاً چند کلمه جلوتر را حدس می‌زدند. اما **کلود کد (Claude Code)** یک **ایجنت ترمینال (Terminal-Native Agent)** است. این ایجنت:

- به فایل‌های واقعی مخزن شما دسترسی محلی دارد و با ابزارهای Read, Edit, Write آن‌ها را ویرایش می‌کند.
- می‌تواند فرامین خط فرمان (Bash, PowerShell) را اجرا کند، لاگ‌های خطا را ببیند و خودش را تصحیح نماید.
- تست‌های پروژه را اجرا می‌کند و تا سبز شدن تست‌ها به چرخه ادامه می‌دهد.
- دارای **دروازه‌های امنیتی مجوز (Permission Gates)** است تا هرگز اقدام مخربی بدون اجازه صریح شما رخ ندهد.

سه قانون طلایی موفقیت در وایب کدینگ

1. **مشخصات و قیود پیش از شروع (Spec & Constraints First):** تمام محدودیت‌ها، تایپ‌ها و انتظارات را در ابتدای چرخه مشخص کنید؛ اصلاح مسیر در میانه کار توکن‌های زیادی را هدر می‌دهد.
2. **چرخه تست قرمز به سبز (Red-to-Green Loop):** ابتدا رفتار سیستم را با تست‌های شکست‌خورده اثبات کنید؛ تست‌ها قطب‌نمای هدایت ایجنت هستند.
3. **شجاعت در آزمایش به لطف چک‌پوینت‌ها (Fearless Iteration):** کلود کد قبل از هر تغییر فایل، چک‌پوینت ایجاد می‌کند. با دستور `/rewind` هر لحظه می‌توانید بدون درگیر شدن با گیت به عقب بازگردید.

فهرست تفصیلی مطالب کتابچه

فصل اول: راه‌اندازی و مبانی بنیادین

۰۱. کلود کد چیست؟ (What Is Claude Code)	مفهوم بنیادین · ۵ دقیقه
۰۲. نصب و راه‌اندازی ابزار خط فرمان (Install the CLI)	کارگاه عملی · ۷ دقیقه
۰۳. اولین مکالمه با ایجنت (Your First Conversation)	شبیه‌ساز مکالمه · ۶ دقیقه
۰۴. چرخه ایجنتیک (The Agentic Loop)	مفهوم بنیادین · ۸ دقیقه
۰۵. حافظه مانا پروژه با CLAUDE.md (Project Memory)	کارگاه عملی · ۱۰ دقیقه

فصل دوم: جریان‌های کاری و قابلیت‌های هسته

۰۶. خواندن و ویرایش فایل‌ها (Reading and Editing Files)	شبیه‌ساز مکالمه · ۷ دقیقه
۰۷. جستجو و کاوش در مخزن کد (Searching the Codebase)	شبیه‌ساز مکالمه · ۶ دقیقه
۰۸. اجرای تست‌ها همراه با کلود (Running Tests with Claude)	شبیه‌ساز مکالمه · ۸ دقیقه
۰۹. عملیات گیت و کامیت‌های معنادار (Git Operations and Commits)	کارگاه عملی · ۷ دقیقه
۱۰. ریفکتورهای چندفایلی (Multi-File Refactors)	شبیه‌ساز مکالمه · ۹ دقیقه
۱۱. دیباگ و خطایابی ساختاریافته (Debugging with Claude)	شبیه‌ساز مکالمه · ۸ دقیقه

فصل سوم: قابلیت‌های پیشرفته و قدرت‌بخش

۱۲. دستورات اسلش توکار (Built-in Slash Commands)	کارگاه عملی · ۸ دقیقه
۱۳. نوشتن دستورات اسلش سفارشی (Custom Slash Commands)	کارگاه عملی · ۷ دقیقه
۱۴. مهارت‌های ایجنت (Agent Skills)	کارگاه عملی · ۹ دقیقه
۱۵. ساب‌ایجنت‌ها و ابزار ایجنت (Subagents & Agent Tool)	مفهوم بنیادین · ۸ دقیقه
۱۶. مود برنامه‌ریزی (Plan Mode)	شبیه‌ساز مکالمه · ۷ دقیقه
۱۷. هوک‌ها: قبل و بعد از ابزارها (Pre & Post Tool Hooks)	کارگاه عملی · ۸ دقیقه
۱۸. سرورهای پروتکل متنی مدل (MCP Servers)	مفهوم بنیادین · ۸ دقیقه

فصل چهارم: الگوها و تکنیک‌های وایب کدینگ

۱۹. الگوی مشخصات → تست‌ها → کد (Spec → Tests → Code)	کارگاه عملی · ۹ دقیقه
۲۰. بهبود تدریجی و گام‌های اتمیک (Iterative Refinement)	شبیه‌ساز مکالمه · ۸ دقیقه
۲۱. چه زمانی مداخله کنیم و چه زمانی بگذاریم اجرا شود؟ (When to Interrupt)	مفهوم بنیادین · ۷ دقیقه
۲۲. کدریویو با کلود (Code Review with Claude)	شبیه‌ساز مکالمه · ۷ دقیقه

۲۳. جریان کار جفت‌برنامه‌نویسی (Pair-Programming Flow)	مفهوم بنیادین 💡 · ۸۰ دقیقه

فصل پنجم: پروژه‌های دنیای واقعی و استقرار

۲۴. ساخت یک فیچر کامل در Next.js (Building a Next.js Feature)	پروژه عملی 🚀 · ۱۰۰ دقیقه
۲۵. رویکرد تست‌محور (Writing Tests-First in Action)	پروژه عملی 🚀 · ۹۰ دقیقه
۲۶. مهاجرت کدهای قدیمی (Migrating Legacy Code)	پروژه عملی 🚀 · ۹۰ دقیقه
۲۷. استقرار و ارسال به پروداکشن (Shipping to Production)	کارگاه عملی 🛠️ · ۸۰ دقیقه
۲۸. پروژه پایانی: ساخت و انتشار یک سایدپروژه (Capstone: Ship a Side Project)	پروژه نهایی 🏆 · ۱۵۰ دقیقه

ضمایم و مراجع تخصصی

ضمیمه شماره ۱: چیت‌شیت جامع و مرجع سریع کلود کد (۱۳ بخش)	چیت‌شیت 🛠️
ضمیمه شماره ۲: واژه‌نامه و اصطلاح‌نامه تخصصی هوش مصنوعی و وایب کدینگ (۸۳ اصطلاح)	دیکشنری 📖

🚀 ماژول SETUP-1

فصل اول: راه‌اندازی و مبانی بنیادین

نصب و راه‌اندازی ابزار خط فرمان، درک عمیق چرخه ایجنتیک (Agentic Loop)، و پیکربندی حافظه مانا پروژه (CLAUDE.md). در پایان این فصل شما کاملاً بر محیط خط فرمان مسلط و آماده تعامل هستید.

درس 01 | مفهوم بنیادین 💡 | ⏱ زمان مطالعه: ۵ دقیقه

۰۱. کلود کد چیست؟ (What Is Claude Code)

ID: cc-01-what-is-claude-code

خلاصه اجرایی: کلود کد (Claude Code) صرفاً یک ابزار تکمیل خودکار کد (Autocomplete) نیست؛ بلکه یک ایجنت هوشمند مستقل است که مستقیماً در ترمینال شما اجرا می‌شود: فایل‌ها را می‌خواند، فرامین سیستم‌عامل را اجرا می‌کند، کدها را ویرایش می‌نماید و برای اقدامات مهم منتظر تایید شما می‌ماند.

تغییر بزرگ ذهنی در وایب کدینگ (Vibe Coding) این است: به جای این‌که ساعت‌ها وقت صرف تایپ کاراکتر به کاراکتر کدها و سر و کله زدن با سینتکس کنید، «نتیجه و هدف نهایی» (Outcomes) را برای ایجنت توصیف می‌کنید. شما در ارتفاع بالاتری از تفکر قرار می‌گیرید؛ شما تمرکزتان را روی «چه چیزی» (What) می‌گذارید و ایجنت به نحو احسن بخش «چگونه» (How) را اجرا می‌کند. کلود کد ابزاری محلی در سیستم‌عامل شماست. این ایجنت با دستورات bash، ابزارهای جستجو مانند grep و glob، و ویرایشگر فایل مستقیماً به مخزن پروژه شما متصل است. اما وجه تمایز حیاتی آن، وجود «دروازه‌های مجوز» (Permission Gates) است؛ یعنی برای هر عملیاتی که ریسک داشته باشد یا تغییری دائمی ایجاد کند، ایجنت توقف کرده و از شما تاییدیه صریح می‌گیرد.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: کدام مورد توصیف دقیق‌تری از کلود کد (Claude Code) است؟

الف یک افزونه IDE که همان‌طور که تایپ می‌کنید کد را تکمیل خودکار می‌کند.

ب یک ایجنت خط فرمان (CLI) که مخزن شما را می‌خواند، دستورات را اجرا می‌کند و با تایید شما کدها را ادیت می‌نماید.

ج یک پلتفرم ابری SaaS که کدهای شما را میزبانی کرده و تست‌ها را آنجا اجرا می‌کند.

د یک ربات بررسی کد (Code Review) که روی پول‌ریکوئست‌های گیت‌هاب کامنت می‌گذارد.

✅ **پاسخ صحیح: گزینه «ب»**
کلود کد به صورت محلی به عنوان یک ابزار خط فرمان روی رایانه شما اجرا می‌شود؛ می‌تواند فایل‌ها را بخواند، دستورات شل را ران کند و کدها را ویرایش نماید، اما هر اقدام پرمخاطره با یک تاییدیه امنیتی متوقف می‌شود.

پرسش 2: تغییر ذهنی و پارادایم بنیادین در «وایب کدینگ» (Vibe Coding) چیست؟

الف سپردن کامل انتشار کدها به هوش مصنوعی بدون هیچ‌گونه نظارت انسانی.

ب تایپ پرامپت‌ها به زبان طبیعی به جای کدنویسی.

ج توصیف نتایج و اهداف نهایی برای ایجنت و قضاوت خروجی‌ها، به جای تمرکز بر تک‌تک کاراکترهای سینتکس.

د سرعت بخشیدن به فرآیند کدنویسی با کپی‌پیست سریع‌تر کدهای تولیدی.

✅ **پاسخ صحیح: گزینه «ج»**
وایب کدینگ یعنی کار کردن در ارتفاع فکری بالاتر: شما تمرکزتان را روی «چه چیزی» (What) و مشخصات سیستم می‌گذارید، و ایجنت هوشمند وظیفه سنگین پیاده‌سازی «چگونه» (How) را انجام می‌دهد.

🧩 اصطلاحات تخصصی این درس:

Agent
A model wired to tools and a loop: it reads, acts, checks the result, and goes again until the task is done or you stop it.

Vibe coding
Building software by describing what you want to an agent and judging the result rather than reading every line of code.

Permission request
The harness pausing before a risky tool call to ask you yes or no; the simplest human-in-the-loop gate there is.

درس 02 | کارگاه عملی 🛠️ | ⏱️ زمان مطالعه: ۷ دقیقه

۰۲. نصب و راه‌اندازی ابزار خط فرمان (Install the CLI)

ID: cc-02-install-the-cli

> **خلاصه اجرایی:** کلود کد را روی رایانه خود راه‌اندازی کرده و صحت عملکرد آن را با اجرای تست اولیه اعتبارسنجی کنید.

برای اجرای کلود کد، نیاز به یک حساب آنتروپیک با پلن فعال (یا کلید API آنتروپیک) دارید. خود بسته خط فرمان کاملاً رایگان نصب می‌شود و تنها هزینه مربوط به مصرف توکن‌های مدل هوش مصنوعی است. توصیه رسمی استفاده از نصاب بومی (Native Installer) از طریق دستور curl در مک/لینوکس یا PowerShell در ویندوز است؛ زیرا نیازی به پیکربندی دستی Node.js ندارد و به‌روزرسانی‌های جدید را خودکار دریافت می‌کند. روش npm نیز برای توسعه‌دهندگانی که ترجیح می‌دهند با زنجیره ابزار جاوااسکریپت کار کنند کاملاً در دسترس است.

> 💡 **نکته کاربردی:** You need a Claude account with a **Pro or Max plan** (or an Anthropic API key). The install itself is free — the agent needs the account to think with.

فرامین نصب برای سیستم‌عامل‌های مختلف:

macOS / Linux:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows:

```
irm https://claude.ai/install.ps1 | iex
```

npm:

```
npm install -g @anthropic-ai/claude-code
```

> 💡 **نکته کاربردی:** Verify with `claude --version` — a version number means the CLI is on your PATH and ready.

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Install the CLI

Pick your platform in the tabs above and run the one-liner. The native installer is the recommended path — no Node.js required.

راستی‌آزمایی: A version number prints. If `command not found`, open a fresh terminal so PATH changes load.

گام 02: Authenticate

Launch Claude Code from any directory. The first run opens a browser window for OAuth login with your Claude account.

راستی‌آزمایی: After login the prompt becomes interactive and shows your working directory in the header.

گام 03: Pick a project

Quit (Ctrl+C twice) and re-run `claude` from inside an existing repo. The directory you launch from becomes the working tree the agent can read and edit.

راستی‌آزمایی: The header shows the project path. Type "hi" to confirm the agent responds.

گام 04: Try a smoke test

Ask the agent to describe your repo. If it can list files and summarize the structure, your install is healthy.

راستی‌آزمایی: You should see a Read tool call against your files, then a 3-line summary.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: کدام پوشه به عنوان دایرکتوری کاری (Working Tree) کلود کد تعیین می‌شود؟

- **الف** پوشه خانگی سیستم کاربر (Home Directory).
- **ب** **پوشه‌ای که دستور claude را درون آن از ترمینال اجرا کرده‌اید.**
- **ج** ریشه درایو اصلی سیستم‌عامل.
- **د** پوشه مخفی .git در پروژه.

✅ **پاسخ صحیح: گزینه «ب»**

دایرکتوری‌ای که دستور claude را در آن فراخوانی می‌کنید، ریشه کاری ایجنت می‌شود و تمامی ابزارهای خواندن، جستجو و ویرایش فایل در همان محدوده فعال خواهند بود.

درس 03 | شبیه‌ساز مکالمه | ⏱ زمان مطالعه: ۶ دقیقه

۰۳. اولین مکالمه با ایجنت (Your First Conversation)

ID: cc-03-first-conversation

> **خلاصه اجرایی:** مشاهده چرخه ایجنتیک در عمل: یک پرامپت، یک فراخوانی ابزار، یک نتیجه، و پاسخ نهایی. این ریتم، نبض تپنده وایب کدینگ است.

هر تعاملی در کلود کد یک چرخه تکرارشونده است. شما با زبان طبیعی سخن می‌گویید. ایجنت ابتدا تحلیل می‌کند که آیا برای پاسخ دقیق نیاز به اطلاعات واقعی از سیستم و کدهای شما دارد یا خیر. اگر پاسخ مثبت باشد، ابزار مناسبی مانند Read یا Grep یا Bash را فراخوانی می‌کند و منتظر دریافت خروجی واقعی می‌ماند. سپس داده‌ها را تحلیل کرده و مجدداً صحبت می‌کند. این حلقه تا پایان کامل کار یا مداخله شما ادامه دارد. نکته بنیادین: ایجنت هرگز حدس نمی‌زند و توهم نمی‌بافد؛ بلکه قبل از خلاصه‌سازی فایل، آن را می‌خواند تا پاسخی مستند و مبتنی بر حقیقت (Grounded Answer) ارائه دهد.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Open the README and tell me what this project does in one sentence.

کلود: I'll read the README first, then summarize.

کلود: It's an interactive curriculum that teaches three agentic coding tools — Claude Code, Codex for mobile, and Antigravity — end to end.

کاربر: Are there any failing tests?

کلود: Let me check by running the test suite.

کلود: Two tests, both passing. The suite is green.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چرا ایجنت قبل از خلاصه‌سازی فایل README، ابتدا ابزار Read را فراخوانی کرد؟

الف چون این کار در پرامپت سیستمی اجباری شده بود.

ب چون برای ارائه پاسخی مستند و دقیق به جای حدس و توهم، به محتوای واقعی فایل نیاز داشت.

ج چون ابزار Read سریع‌تر از حافظه درونی مدل است.

د چون ایجنت همیشه قبل از هر کاری باید فایل README.md را بخواند.

✅ **پاسخ صحیح: گزینه «ب»**

ایجنت بر خلاف مدل‌های سنتی حدس نمی‌زند؛ بلکه با فراخوانی ابزار Read، داده‌های مستند و زنده را از دیسک واکشی کرده و پاسخی کاملاً بر پایه واقعیت (Grounded) تولید می‌کند.

پرسش 2: حلقه تکرارشونده در این مکالمه چگونه بود؟

الف کاربر → خواندن فایل → پایان کار.

ب کاربر → تصمیم ایجنت → فراخوانی ابزار → نتیجه ابزار → پاسخ نهایی ایجنت.

ج ایجنت → کاربر → ابزار → اتمام.

د ابزار → کاربر → ایجنت → ابزار.

✅ **پاسخ صحیح: گزینه «ب»**

چرخه استاندارد ایجنتیک شامل: دریافت پرامپت کاربر، تحلیل نیاز به ابزار، اجرای ابزار و دریافت نتیجه، و سرانجام فرموله‌سازی پاسخ نهایی است.

📚 **اصطلاحات تخصصی این درس:**

Agent
A model wired to tools and a loop: it reads, acts, checks the result, and goes again until the task is done or you stop it.

Tool
A named capability the harness lets the model invoke, such as reading a file, running a shell command or fetching a page.

Tool call
The model's request to run a tool: a structured message naming the tool and its arguments, which the harness then executes.

درس 04 | مفهوم بنیادین 💡 | ⏱ زمان مطالعه: ۸ دقیقه

۰۴. چرخه ایجنتیک (The Agentic Loop)

ID: cc-04-the-agentic-loop

> **خلاصه اجرایی:** ریتم چهارگانه: برنامه‌ریزی (Plan) → اقدام (Act) → مشاهده (Observe) → تنظیم (Adjust). شناخت این چرخه مرز میان جنگیدن با ایجنت و همراهی روان با آن است.

هر اقدامی که ایجنت انجام می‌دهد از ریتم چهارضرب پی‌رویی می‌کند: ۱. برنامه‌ریزی (Plan): تعیین گام بعدی بر اساس هدف نهایی و زمینه فعلی. ۲. اقدام (Act): فراخوانی یک ابزار — خواندن فایل، اجرای دستور شل، یا ویرایش کد. ۳. مشاهده (Observe): خواندن دقیق خروجی ابزار و ارزیابی نتیجه. ۴. تنظیم (Adjust): به‌روزرسانی طرح اجرایی بر اساس بازخورد و تکرار چرخه. وقتی این ریتم را درک کنید، دو اتفاق مهم می‌افتد: نخست این‌که وسواس مداخله‌های ناگهانی در میانه کار را کنار می‌گذارید و اجازه می‌دهید ایجنت خروجی را مشاهده کند؛ دوم این‌که یاد می‌گیرید تمامی قیود و شرایط حیاتی را از همان ابتدا و قبل از آغاز چرخه تعیین کنید (Front-loading Constraints)، زیرا بازتعریف قیود در میانه کار، هزینه بازبرنامه‌ریزی سنگینی به همراه دارد.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Add a /health endpoint to the API. It should return {status: 'ok'} and have a test.

کلود: Plan: find the API entry point, add the route, add a test. Acting now.

کلود: I see Express. Adding the /health route and a Jest test next to existing tests.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: کدام توالی یک ضرب از چرخه ایجنتیک (Agentic Loop) را توصیف می‌کند؟

- **الف** برنامه‌ریزی (Plan) → اقدام (Act) → مشاهده (Observe) → تنظیم (Adjust).
- **ب** اقدام → تست → کامیت → پوش.
- **ج** پرامپت → پاسخ → اصلاح → خروج.
- **د** فکر کردن → کدنویسی → بازبینی → تکرار.

✅ **پاسخ صحیح: گزینه «الف»**

ریتم ۴ گانه چرخه ایجنتیک شامل: تعیین گام بعدی (Plan)، فراخوانی ابزار (Act)، خواندن خروجی ابزار (Observe)، و به‌روزرسانی طرح اجرایی (Adjust) است.

پرسش 2: چرا تعریف زودهنگام قیود (Front-loading Constraints) در ابتدای پرامپت ارزان‌تر از اضافه کردن آن‌ها در میانه کار است؟

- **الف** چون مدل‌های زبانی فقط پرامپت اول را به خاطر می‌سپارند.
- **ب** **چون تغییر قید در میانه کار، ایجنت را مجبور به بازبرنامه‌ریزی، حذف کدهای نوشته‌شده و اتلاف توکن‌های فراوان می‌کند.**
- **ج** چون پرامپت‌های اولیه تخفیف مصرف توکن دارند.
- **د** چون اضافه کردن قید در میانه کار نشست را ریست می‌کند.

✅ **پاسخ صحیح: گزینه «ب»**

قیودی که از ابتدا مشخص باشند از گام اول در مسیر صحیح برنامه‌ریزی می‌شوند؛ اما قیود ناگهانی باعث شکست چرخه فعلی و هدررفت شدید زمان و توکن می‌شوند.

📚 **اصطلاحات تخصصی این درس:**

Agent

A model wired to tools and a loop: it reads, acts, checks the result, and goes again until the task is done or you stop it.

درس 05 | کارگاه عملی 🛠️ | ⏱️ زمان مطالعه: ۱۰ دقیقه

۰۵. حافظه مانا پروژه با CLAUDE.md (Project Memory)

ID: cc-05-claude-md-and-memory

خلاصه اجرایی: دستورالعمل‌های سطح پروژه که ایجنت به صورت خودکار بارگذاری می‌کند؛ پرقدرت‌ترین و اثربخش‌ترین اهرم مخزن در متدولوژی وایب کدینگ.

هر بار که کلود کد را در پوشه‌ای که حاوی فایل CLAUDE.md است باز می‌کنید، این فایل به صورت خودکار در کانتکست ایجنت تزریق می‌شود. این فایل نقش «حافظه مانا و قرارداد کاری» پروژه را بازی می‌کند. یک فایل CLAUDE.md عالی باید کوتاه، تلگرافی و بسیار دقیق باشد. چیزهایی که ایجنت خودش با خواندن package.json متوجه می‌شود (مانند فریم‌ورک‌ها) را حذف کنید؛ در عوض چیزهایی را بنویسید که ایجنت هرگز خودش نمی‌تواند حدس بزند: قراردادهای معماری خاص تیم، نحوه اجرای تست‌ها و ساخت، پیچیدگی‌های استقرار، و اشتباهاتی که قبلاً ایجنت مرتکب شده و شما تذکر داده‌اید.

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Run /init in a real project

.From inside a project with code, type /init. The agent reads your repo and proposes a CLAUDE.md

راستی‌آزمایی: .'A CLAUDE.md appears at your project root with sections like 'Stack' and 'Conventions

گام 02: Trim aggressively

Open the generated file. Delete anything obvious from package.json (frameworks, scripts). Keep only: hidden .conventions, deploy quirks, and rules you've corrected the agent on before

راستی‌آزمایی: .Your CLAUDE.md is under ~30 lines and every line says something the agent could not infer

گام 03: Test that it took

.Quit and relaunch claude in the same directory. Ask a question whose answer requires the file

راستی‌آزمایی: .The agent answers "pnpm — npm install would corrupt the workspaces" without reading any other file

گام 04: Add a personal layer

Create ~/.claude/CLAUDE.md for cross-project preferences (your tone, your default test framework). It loads on .every project, layered under the project file

راستی‌آزمایی: .These rules now apply in every Claude Code session you open

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چه مطالبی باید درون فایل CLAUDE.md قرار گیرند؟

الف کپی کامل تمام فایل‌های package.json و توضیحات معماری کلی وب.

ب دستورات بیلد، تست، قراردادهای مخفی پروژه، و قواعدی که قبلاً ایجنت را به خاطر نقض آن‌ها اصلاح کرده‌اید.

ج تاریخچه لاگ‌های کامل گیت و اسامی تمام مشارکت‌کنندگان.

د لیست تمام فایل‌های موجود در ریپازیتوری.

✅ **پاسخ صحیح: گزینه «ب»**

فایل CLAUDE.md باید خلاصه و تلگرافی باشد و فقط اطلاعاتی را شامل شود که ایجنت خودش نمی‌تواند از روی کدهای منبع بفهمد.

پرسش 2: لایه تنظیمات و دستورالعمل‌های شخصی کاربر در چه مسیری ذخیره می‌شود؟

الف در فایل `~/.claude/CLAUDE.md` در پوشه خانگی کاربر.

ب در پوشه `.git/config` پروژه.

ج در متغیرهای سیستمی ویندوز یا بش.

د در رجیستری سیستم‌عامل.

✅ **پاسخ صحیح: گزینه «ب»**

فایل `~/.claude/CLAUDE.md` در دایرکتوری خانگی کاربر قرار دارد و تنظیمات شخصی او را در تمام پروژه‌ها زیر لایه تنظیمات پروژه تزریق می‌کند.

📚 **اصطلاحات تخصصی این درس:**

AGENTS.md

The instructions file at the repo root that every session reads first: .commands, conventions, gotchas. CLAUDE.md in Claude Code

Agent

A model wired to tools and a loop: it reads, acts, checks the result, .and goes again until the task is done or you stop it

Slash command

A typed /name shortcut that runs a built-in action or expands a .saved prompt, so the prompts you reuse live in the repo

ماژول CORE-WORKFLOWS-2

فصل دوم: جریان‌های کاری و قابلیت‌های هسته

جریان‌های کاری و ریتم‌های روزمره‌ای که کار با ایجنت را از تایپ کردن دستی کاراکترها سریع‌تر، مطمئن‌تر و لذت‌بخش‌تر می‌کنند: خواندن و ویرایش، جستجو، اجرای تست‌ها، کامیت‌های گیت، ریفکتورهای چندفایلی و دیباگ اصولی.

درس 06 | شبیه‌ساز مکالمه | زمان مطالعه: ۷ دقیقه

۰۶. خواندن و ویرایش فایل‌ها (Reading and Editing Files)

ID: cc-06-reading-and-editing

خلاصه اجرایی: ابزار Read فایل‌ها را در حافظه بارگذاری می‌کند، ابزار Edit رشته‌های مشخص را جایگزین می‌نماید، و Write فایل جدید ایجاد یا بازنویسی می‌کند.

ایجنت ماهر از انضباط کاری سفت‌وسختی پیروی می‌کند: قبل از هرگونه ویرایش، فایل را به دقت می‌خواند؛ در ویرایش‌ها کمترین مقدار لازم را تغییر می‌دهد تا ریسک شکست ساختار کد به حداقل برسد؛ و به جای حدس زدن شماره خطوط، با تطابق رشته‌ای منحصربه‌فرد (Exact String Matching) تغییرات را اعمال می‌کند. درک این انضباط به شما کمک می‌کند پرامپت‌هایی بنویسید که محدوده دقیق تغییرات را مشخص کنند و از بازنویسی غیرضروری کل فایل‌ها ممانعت به عمل آورید.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Rename the variable `usr` to `user` in src/auth.ts. Update every usage.

کلود: I'll read the file first to see all the usages, then make a single Edit.

کلود: Three usages. Renaming all three with replace_all.

کلود: Done. Verified by re-reading: all three usages now say `user`.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چرا ایجنت پس از ویرایش، مجدداً فایل را با ابزار Read بازخوانی کرد؟

الف چون تغییرات به درستی در فایل ذخیره نشده بودند.

ب **برای اطمینان از این‌که جایگزینی دقیقاً همان‌گونه که انتظار می‌رفت اعمال شده و محتوای مجاور آسیبی ندیده است.**

ج چون ابزار Edit خروجی تایید بازنمی‌گرداند.

د چون پرامپت سیستمی پس از هر ویرایش بازخوانی را اجباری کرده است.

✅ **پاسخ صحیح: گزینه «ب»**

انضباط مهندسی ایجنت ایجاب می‌کند که با بازخوانی فایل، خروجی ویرایش را مشاهده (Observe) کرده و صحت ساختار کد را تایید کند.

🧩 اصطلاحات تخصصی این درس:

Agent

A model wired to tools and a loop: it reads, acts, checks the result, and goes again until the task is done or you stop it.

درس 07 | شبیه‌ساز مکالمه | ⏱ زمان مطالعه: ۶ دقیقه

۰۷. جستجو و کاوش در مخزن کد (Searching the Codebase)

ID: cc-07-searching-the-codebase

خلاصه اجرایی: ابزار Glob پاسخ این است که «فلان فایل‌ها در کجا زندگی می‌کنند» و ابزار Grep پاسخ می‌دهد که «چه کسی از فلان متغیر استفاده کرده است».

یکی از بزرگ‌ترین اشتباهات در کار با مدل‌های زبانی، تلاش برای کپی کردن کل پایگاه کد درون پنجره متنی است. این کار کانتکست را پر کرده و دقت مدل را کاهش می‌دهد. کلود کد با استفاده از Glob و Grep، به صورت جراحی‌گونه و دقیق عمل می‌کند. تنها بخش‌هایی که مستقیماً به تسک جاری مربوط می‌شوند کشف و بارگذاری می‌گردند؛ این یعنی سرعت بالاتر، عدم توهم و صرفه‌جویی چشم‌گیر در مصرف توکن.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Where is the auth middleware applied? Show me every route that uses it.

کلود: I'll grep for the middleware import and its uses.

کلود: It's applied in two places: globally on /api/private (server.ts:18) and on the admin router (admin.ts:7). Every other route is public.

سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چه زمانی باید از ابزار Glob به جای Grep استفاده کنید؟

- **الف** وقتی می‌خواهید بدانید چه فایل‌هایی با یک الگوی نام‌گذاری مشخص وجود دارند (مثلاً پیدا کردن تمام فایلهای تست).
- **ب** وقتی می‌خواهید محتوای متنی درون یک خط از کد را جستجو کنید.
- **ج** وقتی حجم فایل‌ها بسیار سنگین است.
- **د** وقتی نیاز به اجرای تست‌های واحد دارید.

پاسخ صحیح: گزینه «الف»

ابزار Glob ساختار درخت فایل‌ها را بر اساس نام می‌کاود («فایل X کجاست؟»)، در حالی که Grep درون محتوای متنی فایل‌ها را جستجو می‌کند («چه کسی از تابع Y استفاده کرده؟»).

درس 08 — شبیه‌ساز مکالمه — زمان مطالعه: ۸ دقیقه

۰۸. اجرای تست‌ها همراه با کلود (Running Tests with Claude)

ID: cc-08-running-tests

خلاصه اجرایی: وقتی تست‌ها تعریف شده باشند، ایجنت می‌تواند چرخه تست قرمز به سبز (Red → Green) را کاملاً خودکار به سرانجام برساند.

مهارت کلیدی در این مرحله «اعتماد به چرخه» (Trusting the Loop) است. نیازی نیست بعد از هر خط ویرایش توسط ایجنت سرک بکشید؛ بگذارید تست‌ها را اجرا کند، لاگ‌های خطا را ببیند و خودش را تصحیح کند تا همه تست‌ها سبز شوند. وظیفه شما این است که تستی مشخص و شفاف با دستور یک‌خطی در اختیار ایجنت بگذارید تا به عنوان قطب‌نمای موفقیت از آن استفاده کند.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Add a test that login() throws if email is missing. Then make sure it passes.

کلود: Adding the test first. It should fail, then I'll fix it.

کلود: Failing as expected. Now adding the validation.

کلود: Green. Test catches missing emails; implementation throws cleanly.

سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: ارزش اجرای تست توسط ایجنت «قبل» از نوشتن پیاده‌سازی اصلی در چیست؟

الف) این کار کدهای قبلی را کش می‌کند تا در اجرای بعدی سریع‌تر شوند.

ب) اثبات می‌کند که تست واقعاً کار می‌کند و به درستی شکست می‌خورد (نه این‌که تستی توخالی باشد که همیشه پاس شود).

ج) خطاهای سینتکسی فایل‌های جاوااسکریپت را برطرف می‌سازد.

د) به ایجنت اجازه می‌دهد بدون نیاز به نوشتن کد کار را تمام کند.

پاسخ صحیح: گزینه «ب»

دیدن شکست تست (Red) اطمینان می‌دهد که تست نسبت به رفتار مورد نظر حساس است و هنگامی که بعداً سبز شد، موفقیت واقعی را نشان می‌دهد.

درس 09 | کارگاه عملی | زمان مطالعه: ۷ دقیقه

۰۹. عملیات گیت و کامیت‌های معنادار (Git Operations and Commits)

ID: cc-09-git-and-commits

خلاصه اجرایی: کلود کد می‌تواند تغییرات را Stage کند، پیام کامیت استاندارد بنویسد و شاخه‌بندی کند؛ اما مدیریت پوش نهایی در دستان شماست.

کلود کد پیش از کامیت زدن، دستورات git diff، git status و git log را بررسی می‌کند تا سبک کامیت‌نویسی پروژه شما را کشف کرده و دقیقاً طبق همان الگو پیام بنویسد. یک قانون طلایی در وایب کدینگ: هرگز اجازه push خودکار را ندهید. Push کردن یک اقدام با وضعیت اشتراکی روی سرور است؛ بگذارید کامیت‌ها به صورت محلی انجام شوند و کنترل ارسال به ریموت را خودتان در دست بگیرید.

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Ask for a commit

.After making changes, ask the agent to commit. It will run status, diff, and log to read the project's commit style first

راستی‌آزمایی: You see Bash calls for `git status`, `git diff`, `git log`, then a commit with a message that matches the repo's tone.

گام 02: Refuse the auto-push

.If the agent offers to push, say no. Push is a shared-state action — keep that on your hands

راستی‌آزمایی: .The session ends with the commit local; you can `git log -1` and inspect

گام 03: Reset gracefully

.If the message or content is wrong, ask the agent to soft-reset and try again, rather than amending

راستی‌آزمایی: .git status` shows the changes back in your working tree, ready to be re-committed`

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چرا دستور `git push` برای ایجنت در دسته‌ای کاملاً متفاوت از `git commit` قرار می‌گیرد؟

الف چون کامیت زدن ارزان‌تر از پوش کردن است.

ب چون push یک اقدام با اثر عمومی و اشتراکی روی ریموت است که شاخه‌های همکاران را تحت تاثیر قرار می‌دهد و باید با نظارت انسان باشد.

ج چون کلود کد ابزاری برای اتصال به اینترنت ندارد.

د چون گیت اجازه کامیت بدون پوش را نمی‌دهد.

✅ **پاسخ صحیح: گزینه «ب»**

کامیت‌ها محلی هستند و به سادگی قابل بازگشت (Reset) می‌باشند؛ اما Push وضعیت عمومی ریپازیتوری را تغییر می‌دهد و نباید خودکار اجرا شود.

📚 اصطلاحات تخصصی این درس:

Agent
A model wired to tools and a loop: it reads, acts, checks the result, and goes again until the task is done or you stop it.

درس 10 | شبیه‌ساز مکالمه | ⏱ زمان مطالعه: ۹ دقیقه

۱۰. ریفکتورهای چندفایلی (Multi-File Refactors)

ID: cc-10-multi-file-refactors

خلاصه اجرایی: بزرگ‌ترین کابوس انسان‌ها در ریفکتورینگ، حساب‌داری تغییرات در صدها فایل است: اینترفیس‌ها، ایمپورت‌ها، فراخوانی‌ها. این دقیقاً همان جایی است که ایجنت می‌درخشد.

الگوی موفق برای ریفکتورهای چندفایلی: در یک جمله شفاف هدف تغییر ساختار را مشخص کنید؛ سپس به ایجنت دستور دهید تمامی محل‌های استفاده (Call Sites) را با Grep بیابد، تک‌تک فایل‌ها را اصلاح نماید، و در نهایت با اجرای بیلد یا تست، اطمینان حاصل کند که هیچ ایمپورت شکسته‌ای باقی نمانده است.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Rename the function `getUser` to `fetchUser` everywhere. Then run tests.

کلود: Finding call sites first.

کلود: Three files. Editing each, then running tests.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چه عاملی به ریفکتورهای چندفایلی با ایجنت اطمینان و قابلیت اتکا می‌بخشد؟

الف استفاده از مدل‌های هوش مصنوعی بزرگ‌تر.

ب پیدا کردن تمامی محل‌های فراخوانی با Grep، ویرایش جراحی‌گونه و تایید نهایی با بیلد یا تست.

ج بازنویسی کامل کل پروژه از صفر.

د حذف تمامی تست‌های قدیمی.

✅ **پاسخ صحیح: گزینه «ب»**

کشف تمام نقاط تماس با ابزارهای جستجو و سپس راستی‌آزمایی با تست یا کامپایلر تایپ‌اسکریپت، از جا ماندن ارجاعات قدیمی جلوگیری می‌کند.

📚 **اصطلاحات تخصصی این درس:**

Automated check

A mechanical pass/fail test the harness or CI runs on the agent's work: types, lint, tests, build. Cheap, fast, no judgement.

درس 11 | شبیه‌ساز مکالمه | ⏱ زمان مطالعه: ۸ دقیقه

۱۱. دیباگ و خطایابی ساختاریافته (Debugging with Claude)

ID: cc-11-debugging

خلاصه اجرایی: اغلب افراد در دیباگ شکست می‌خورند چون حدس می‌زنند! الگوی ساختاریافته کلود کد: طرح فرضیه → آزمون با لاگ یا تست → تایید فرضیه → اعمال اصلاح.

ایجنت‌ها اگر رها شوند ممکن است در حل باگ به کدهای پیچیده و وصله‌پینه‌ای روی بیاورند. فرمول هدایت ایجنت در دیباگ: «من فکر می‌کنم علت باگ در بخش X است؛ قبل از هر دستکاری، یک لاگ یا تست شکست‌خورده برای اثبات آن بنویس؛ اگر فرضیه تایید شد، ریشه‌ای‌ترین راهکار را پیاده‌سازی کن».

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: The /users endpoint returns 500 sometimes. Find the cause structurally — hypothesis, evidence, fix.

کلود: Reading the handler first.

کلود: Hypothesis: when the user doesn't exist, findById returns null and `user.toJSON()` throws. Adding a failing test.

کلود: Hypothesis confirmed. Adding the null check + 404.

سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چرا نوشتن یک تست شکست‌خورده قبل از برطرف کردن باگ ارزش بالایی دارد؟

الف) تست‌ها باعث می‌شوند ایجنت کد کمتری بنویسد.

ب) وجود باگ را به طور قطعی اثبات می‌کند و به عنوان محافظی در برابر بازگشت مجدد باگ (رگرسیون) در آینده عمل می‌نماید.

ج) تعداد کامیت‌های پروژه را در گیت افزایش می‌دهد.

د) مدت زمان دیباگ را کوتاه‌تر می‌کند چون تست‌ها نیازی به اجرا ندارند.

پاسخ صحیح: گزینه «ب»

تست ناموفق باگ را بازتولید می‌کند؛ وقتی باگ رفع شود تست سبز می‌شود و در آینده نیز مانع از بازگشت باگ خواهد شد.

🛠️ ماژول POWER-FEATURES-3

فصل سوم: قابلیت‌های پیشرفته و قدرت‌بخش

دستورات اسلش توکار، اسلش کامندهای اختصاصی، مهارت‌های ایجنت (Agent Skills)، ساب‌ایجنت‌ها، مود برنامه‌ریزی (Plan Mode)، هوک‌های قبل و بعد از ابزار (Hooks) و سرورهای MCP — جعبه‌ابزاری که کلود کد را مطابق میل شما منعطف می‌سازد.

درس 12 | کارگاه عملی 🛠️ | ⏱️ زمان مطالعه: ۸ دقیقه

۱۲. دستورات اسلش توکار (Built-in Slash Commands)

ID: cc-12-slash-commands

> **خلاصه اجرایی:** دستورات میانبر حیاتی: /cost, /doctor, /model, /rewind, /compact, /resume, /clear, /init. شناخت این‌ها مرز بین گیج شدن و هدایت مقتدرانه است.

دستورات اسلش میانبرهای درجه یک در محیط کلود کد هستند: - دستور /init: پروژه را اسکن کرده و CLAUDE.md پایه را تولید می‌کند. - دستور /clear: نشست جاری را کاملاً پاک می‌کند اما حافظه CLAUDE.md را حفظ می‌نماید؛ برای شروع تسک جدید ضروری است. - دستور /compact: نشست طولانی را خلاصه می‌کند تا حافظه کانتکست آزاد شود بدون این‌که پیوستگی کار از بین برود. - دستور /rewind (یا دوبار زدن کلید Escape): ماشین زمان شماست! کدهای تغییریافته یا مکالمه را به چک‌پوینت‌های قبلی برمی‌گرداند بدون درگیر شدن با گیت. - دستور /model: سوئیچ سریع بین مدل‌ها (مثلاً استفاده از Opus برای طراحی معماری و سپس استفاده از Haiku برای ادیت‌های سریع).

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Try /init

.Generates or updates CLAUDE.md by reading your repo. Use it once per project, then prune the output

راستی‌آزمایی: .A CLAUDE.md is written or updated at the project root

گام 02: Use /clear when context drifts

clear wipes the conversation but keeps your project memory. Run it when you switch tasks — fresh context/ .produces sharper plans

راستی‌آزمایی: .The chat starts a new session; CLAUDE.md is still loaded

گام 03: Resume a long session with /resume

.If you closed Claude Code and want to pick up where you left off, /resume loads the previous session

راستی‌آزمایی: .The agent restates the goal of the prior session before you continue

گام 04: Compact when you must keep going

compact summarizes the conversation so far so context doesn't blow up. Less destructive than /clear when you/ .need continuity

راستی‌آزمایی: .The session continues with a much smaller context window than before

گام 05: Rewind when a change goes sideways

Claude Code checkpoints your files before each change. /rewind (or double-tap Escape) opens the checkpoint picker .— restore the code, the conversation, or both. It's the undo button that makes bold experiments cheap

راستی‌آزمایی: .A picker lists recent checkpoints; selecting one restores that state without touching your git history

گام 06: Match the model to the task

model switches models mid-session. Reach for the frontier tier (Opus 4.8, or the Claude 5 family where available)/ .on hard refactors and planning; drop to Haiku 4.5 for fast, cheap iteration on simple edits

راستی‌آزمایی: .The picker shows the active model; the status line reflects your choice on the next turn

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: تفاوت میان دستورات `/clear` و `/compact` در چیست؟

الف هیچ تفاوتی ندارند و نام مستعار یکدیگرند.

ب دستور /clear کل گفتگو را پاک می‌کند؛ در حالی که /compact آن را خلاصه کرده تا فضا آزاد شود اما پیوستگی کار حفظ گردد.

ج دستور /clear ابزارها را فعال نگه می‌دارد اما /compact آن‌ها را غیرفعال می‌سازد.

د دستور /compact سریع‌تر از /clear اجرا می‌شود.

✅ **پاسخ صحیح: گزینه «ب»**

دستور /clear برای شروع یک تسک کاملاً جدید است؛ اما /compact برای ادامه‌دادن کارهای طولانی با کانتکست بهینه‌شده استفاده می‌شود.

پرسش 2: دستور `/rewind` چه چیزی را بازمی‌گرداند؟

الف فقط تاریخچه پیام‌های چت را.

ب یک چک‌پوینت از کدها، مکالمه یا هر دو را که قبل از هر تغییر به صورت خودکار ثبت شده است.

ج آخرین کامیت گیت را.

د تنها فایل‌های حذف‌شده را.

✅ **پاسخ صحیح: گزینه «ب»**

دستور /rewind ماشین زمان کلود کد است و به شما اجازه می‌دهد کدهای تغییریافته یا کانتکست چت را بدون دستکاری گیت به حالت قبل برگردانید.

📚 اصطلاحات تخصصی این درس:

Clearing
Wiping the session so the next request starts from an empty context window, keeping only the standing instructions.

Context
Everything the agent currently has in front of it that bears on the task: loaded files, your messages, tool results so far.

Model
The trained network itself: billions of parameters that turn a context into the next token, and nothing more.

Session
One continuous run with an agent, from an empty context to the moment you clear it, close it, or hand its work off.

Compaction
Replacing the session so far with a shorter summary of it, freeing the context window at the cost of detail.

Slash command
A typed /name shortcut that runs a built-in action or expands a saved prompt, so the prompts you reuse live in the repo.

درس 13 | کارگاه عملی 🛠️ | ⏱️ زمان مطالعه: ۷ دقیقه

۱۳. نوشتن دستورات اسلش سفارشی (Custom Slash Commands)

ID: cc-13-custom-slash-commands

> **خلاصه اجرایی:** پرامپت‌های پرتکرار تیم را به دستوراتی تک‌کلمه‌ای با یک کلید تبدیل کنید و آن‌ها را در پوشه `/claude/commands.` نگه‌داری نمایید.

با ایجاد فایل‌های مارک‌داون در مسیر `/claude/commands.`، می‌توانید فرامینی مانند `review.md/` یا `fix-issue.md/` بسازید. این فایل‌ها می‌توانند متغیرهای آرگومان `ARGUMENTS$` را دریافت کرده و یک پروسه استاندارد و مهندسی‌شده را برای کل تیم تضمین کنند.

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Create the commands folder

Inside your repo, create the directory if it doesn't exist.

راستی‌آزمایی: .claude/commands/ exists at your project root.

گام 02: Author the command

Write a markdown file with the prompt you want fired. Use $ARGUMENTS for inline arguments.

راستی‌آزمایی: .claude/commands/review-pr.md exists with that content.

گام 03: Use it

From any session in that repo, type the command.

راستی‌آزمایی: The agent runs the prompt with 'auth module' substituted into $ARGUMENTS.

گام 04: Iterate

Run the command. If the output is too long, too short, or wrong — edit the markdown and re-run. Slash commands evolve like any other prompt.

راستی‌آزمایی: After edits, the new behavior is live on the next invocation. No reload needed.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: دستورات اسلش سفارشی (Custom Slash Commands) در چه مسیری ذخیره می‌شوند؟

الف در فایل `.claude/config.json`.

ب در مسیر `.claude/commands/` به صورت فایل‌های مارک‌داون (`.md`).

ج در ریشه پروژه در فایل CLAUDE.md.

د در تنظیمات گلوبال کاربری سیستم‌عامل.

✅ **پاسخ صحیح: گزینه «ب»**

هر فایل مارک‌داون درون مسیر `.claude/commands/` مستقیماً به یک دستور اسلش جدید به نام `/` تبدیل می‌شود.

📚 **اصطلاحات تخصصی این درس:**

Slash command

A typed /name shortcut that runs a built-in action or expands a saved prompt, so the prompts you reuse live in the repo.

درس 14 | کارگاه عملی 🛠️ | ⏱️ زمان مطالعه: ۹ دقیقه

۱۴. مهارت‌های ایجنت (Agent Skills)

ID: cc-28-agent-skills

خلاصه اجرایی: مهارت یا Skill پوشه‌ای است شامل یک فایل SKILL.md، به همراه اسکریپت‌ها، قالب‌ها و راهنماها؛ روش استاندارد آموزش رفتارهای جدید به ایجنت.

اسکیل‌ها زمانی به کار می‌آیند که می‌خواهید ایجنت در یک دامنه خاص (مثل نوشتن کوئری بهینه برای دیتابیس، تبدیل کدهای فیگما به تیل‌ویند، یا بررسی چک‌لیست سئو) با تخصص کامل و متودولوژی خاص عمل کند. اسکیل‌ها به صورت خودکار با برچسب‌های متنی و نیازمندی‌های پرامپت فعال می‌شوند.

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Scaffold a skill

.Project skills live in .claude/skills//SKILL.md. Personal ones live in ~/.claude/skills/ and follow you across projects

راستی‌آزمایی: .The folder exists at your project root

گام 02: Write SKILL.md

Frontmatter carries the name and — most importantly — the description Claude matches against. The body is the .playbook

راستی‌آزمایی: .SKILL.md exists with frontmatter and numbered instructions

گام 03: Trigger it naturally

.Don't type a slash command — just ask in plain language and watch Claude pick the skill up

راستی‌آزمایی: .The transcript shows the skill loading, then the playbook being followed step by step

گام 04: Tune the description

If the skill didn't fire, the description didn't match how you actually ask. Rewrite it with the phrases you'd really use — .that line is the routing layer

راستی‌آزمایی: .The same natural-language ask now reliably triggers the skill

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: تفاوت کلیدی میان یک دستور اسلش (Slash Command) و یک مهارت ایجنت (Agent Skill) در چیست؟

- **الف** دستورات اسلش سریع‌تر هستند اما اسکیل‌ها کندترند.
- **ب** **دستور اسلش صریحاً توسط کاربر با تایپ کاراکتر / فراخوانی می‌شود؛ در حالی که مهارت به صورت خودکار توسط ایجنت بر اساس محتوای پرامپت فعال می‌گردد.**
- **ج** اسکیل‌ها فقط در زبان پایتون کار می‌کنند.
- **د** هیچ تفاوتی ندارند.

✅ **پاسخ صحیح: گزینه «ب»**
اسکیل‌ها مانند ابزارهای شناختی در جعبه‌ابزار ایجنت هستند که وقتی تسک نیازمند تخصص خاصی باشد، خود ایجنت آن‌ها را لود و اجرا می‌کند.

پرسش 2: چرا داشتن ۳۰ مهارت مختلف در پروژه باعث اشغال و آلودگی پنجره کانتکست نمی‌شود؟

- **الف** چون اسکیل‌ها با الگوریتم Zip فشرده می‌شوند.
- **ب** **چون ایجنت در ابتدا تنها متادیتا و توضیحات کوتاه اسکیل‌ها را در حافظه دارد و فقط زمانی که به مهارت نیاز باشد متن کامل آن را لود می‌کند.**
- **ج** چون اسکیل‌ها خارج از حافظه مدل زبانی اجرا می‌گردند.
- **د** چون سقف مصرف توکن برای اسکیل‌ها رایگان است.

✅ **پاسخ صحیح: گزینه «ب»**
بارگذاری تنبل (Lazy Loading): فقط توصیف یک‌خطی مهارت‌ها در کانتکست است؛ محتوای کامل SKILL.md تنها در صورت تطابق با تسک بارگذاری می‌گردد.

📚 اصطلاحات تخصصی این درس:

Skill
A named, reusable procedure the agent loads on demand: a folder of instructions and optional scripts, used when a task matches.

Slash command
A typed /name shortcut that runs a built-in action or expands a saved prompt, so the prompts you reuse live in the repo.

Progressive disclosure
Giving the agent a short index up front and letting it load the detail only when a task calls for it.

درس 15 | مفهوم بنیادین 💡 | ⏱ زمان مطالعه: ۸ دقیقه

۱۵. ساب‌ایجنت‌ها و ابزار ایجنت (Subagents & Agent Tool)

ID: cc-14-subagents

> **خلاصه اجرایی:** یک ساب‌ایجنت، یک نشست تازه و موازی از کلود است که توسط ایجنت اصلی برای انجام کاری مشخص و متمرکز متولد می‌شود.

مشکل کارهای بزرگ در هوش مصنوعی این است که لاگ‌های پرحجم و جستجوهای پراکنده، کانتکست ایجنت اصلی را آلوده و کند می‌کنند. ساب‌ایجنت در یک کانتکست ایزوله متولد می‌شود، تمام جستجوها و آزمایش‌ها را انجام می‌دهد، و صرفاً یک «چکیده خالص و نتیجه نهایی» به ایجنت اصلی برمی‌گرداند.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چه زمانی استفاده از یک ساب‌ایجنت (Subagent) تصمیم درستی است؟

الف) برای ویرایش فایل‌های تک‌خطی ساده.

ب) برای انجام جستجوهای عمیق، خواندن دهها فایل و آزمون‌های پرتکرار که ممکن است کانتکست ایجنت اصلی را آلوده و شلوغ کنند.

ج) تنها زمانی که اینترنت قطع است.

د) برای کامیت زدن به گیت.

✅ **پاسخ صحیح: گزینه «ب»**
ساب‌ایجنت کار کثیف و پرحجم جستجو را در یک اتاق تمیز و مجزا انجام می‌دهد و فقط عصاره و نتیجه نهایی را به جلسه اصلی بازمی‌گرداند.

پرسش 2: ساب‌ایجنت چه چیزی را به ایجنت اصلی بازمی‌گرداند؟

الف) کل تاریخچه تمامی توکن‌ها و لاگ‌های اجراشده‌اش را.

ب) یک پاسخ خلاصه‌شده و متمرکز که هدف تسک تفویض‌شده را مستقیماً پاسخ می‌دهد.

ج) فایل‌های سیستم‌عامل را در قالب آرشیو tar.

د) هیچ خروجی برنمی‌گرداند و کار را در پس‌زمینه خاتمه می‌دهد.

✅ **پاسخ صحیح: گزینه «ب»**
ساب‌ایجنت یک خلاصه استخراج‌شده و نهایی از پاسخ تسک برمی‌گرداند؛ بنابراین کانتکست ایجنت اصلی سبک و شفاف باقی می‌ماند.

اصطلاحات تخصصی این درس:

Subagent
A separate agent the main agent spins up for a bounded task, with its own context window; only its final report comes back.

Agent
A model wired to tools and a loop: it reads, acts, checks the result, and goes again until the task is done or you stop it.

Context
Everything the agent currently has in front of it that bears on the task: loaded files, your messages, tool results so far.

Context window
The fixed-size buffer of tokens a model can read in one request; if something isn't in it, the model can't see it.

System prompt
The standing instructions the harness puts in front of every request: who the agent is, what tools it has, how it should behave.

درس 16 | شبیه‌ساز مکالمه | زمان مطالعه: ۷ دقیقه

۱۶. مود برنامه‌ریزی (Plan Mode)

ID: cc-15-plan-mode

خلاصه اجرایی: در مود برنامه‌ریزی، ایجنت اجازه خواندن، جستجو و استدلال دارد، اما حق دستکاری کدها یا اجرای کامندهای مخرب را ندارد.

کلید میانبر Shift+Tab شما را بین مود معمولی و مود Plan جابه‌جا می‌کند (یا از طریق فلگ `--permission-mode plan`). در این مود، ایجنت مخزن را شخم می‌زند و یک نقشه راه گام‌به‌گام و تاییدپذیر ارائه می‌دهد. بعد از توافق بر سر پلن، مود را تغییر می‌دهید تا اجرای سریع آغاز شود.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Migrate our auth from session cookies to JWTs. Don't change anything yet — propose the plan [Plan mode].

کلود: Reading current auth implementation first.

کلود: Plan: 1. Add JWT issue/verify utilities (src/auth/jwt.ts). 2. Replace session middleware with JWT middleware; keep req.user shape stable. 3. Update 6 call sites that read req.session — they should switch to req.user. 4. Add tests for: valid token, expired token, missing token, malformed. 5. Migration: keep old session middleware behind a flag for one release. No edits yet. Approve to proceed.

کاربر: Approved. Go.

کلود: Exiting plan mode. Acting now.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چرا در مود برنامه‌ریزی (Plan Mode) ویرایش فایل‌ها ممنوع است؟

الف چون مدل‌های هوش مصنوعی توانایی نوشتن در این مود را ندارند.

ب تا ایجنت قبل از دستکاری کدهای سیستم، فرصت کاوش عمیق، درک معماری و جلب رضایت شما روی نقشه راه را پیدا کند.

ج برای صرفه‌جویی در فضای هارد دیسک.

د چون گیت در مود پلن قفل می‌شود.

✅ **پاسخ صحیح: گزینه «ب»**

مود پلن فضایی امن برای تفکر خالص و تحلیل ریسک است بدون این‌که نگران شکستن کدها قبل از توافق نهایی باشید.

🧠 **اصطلاحات تخصصی این درس:**

Plan mode
A read-only agent mode: the agent may search and read, but must propose a plan for your approval before it edits anything.

Session
One continuous run with an agent, from an empty context to the moment you clear it, close it, or hand its work off.

درس 17 | کارگاه عملی 🛠️ | ⏱️ زمان مطالعه: ۸ دقیقه

۱۷. هوک‌ها: قبل و بعد از ابزارها (Pre & Post Tool Hooks)

ID: cc-16-hooks

خلاصه اجرایی: هوک‌ها به شما اجازه می‌دهند منطق سفارشی خود را درون حلقه ایجنت تزریق کنید: اجرای اجباری لینتر یا جلوگیری از دستورات پرخطر.

دو نوع هوک اصلی داریم: - PreToolUse: پیش از اجرای یک ابزار شل یا ادیت فعال می‌شود؛ اگر اسکریپت خروجی غیر صفر (مثل exit 2) برگرداند، عملیات ایجنت متوقف شده و به کاربر گزارش می‌شود. - PostToolUse: بلافاصله پس از ادیت فایل اجرا می‌شود؛ مثلاً اجرای خودکار prettier یا eslint تا کدهای تولیدی همیشه استاندارد و تمیز بمانند.

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Open the settings file

.Create or open the project settings

راستی‌آزمایی: .An empty or existing settings.json opens

گام 02: Add a post-edit format hook

.Run prettier on every Edit/Write to keep code consistent without asking the agent

راستی‌آزمایی: .After the agent edits a file, you see prettier run in the tool stream

گام 03: Add a pre-bash safety hook

.Block the agent from running risky commands without asking. The hook intercepts and you decide

راستی‌آزمایی: .`Bash calls now route through your script. Add a deny list for things like `rm -rf /` and `git push --force

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: تنظیمات هوک‌ها (Hooks) در کجا تعریف و نگه‌داری می‌شوند؟

الف در کرنل سیستم‌عامل.

ب در فایل تنظیمات کلود کد در مسیر `.claude/config.json` یا تنظیمات پروژه تحت کلید hooks.

ج در داخل کدهای سورس برنامه.

د در متغیرهای فایل .env.

✅ **پاسخ صحیح: گزینه «ب»**

هوک‌ها درون تنظیمات پیکربندی کلود کد قرار می‌گیرند و به ابزارها و الگوهای دستورات متصل می‌شوند.

📚 اصطلاحات تخصصی این درس:

Agent
A model wired to tools and a loop: it reads, acts, checks the result, and goes again until the task is done or you stop it.

Hooks
Commands you configure the harness to run at fixed moments, such as before or after a tool call, regardless of what the model wants.

Tool call
The model's request to run a tool: a structured message naming the tool and its arguments, which the harness then executes.

درس 18 | مفهوم بنیادین 💡 | ⏱️ زمان مطالعه: ۸ دقیقه

۱۸. سرورهای پروتکل متنی مدل (MCP Servers)

ID: cc-17-mcp-servers

خلاصه اجرایی: پروتکل MCP (Model Context Protocol) استاندارد باز برای اتصال کلود به دیتابیس‌ها، مرورگرها، مستندات، ایشیوهای گیت‌هاب و سرویس‌های خارجی است.

ابزارهای توکار کلود کد، به ایجنت یک ترمینال روی رایانه شما می‌دهند؛ اما سرورهای MCP به ایجنت دسترسی به کل اینترنت، دیتابیس‌های Postgres و Mongo، ایشیوهای گیت‌هاب و جیرا، و حتی کنسول‌های ابری را اهدا می‌کنند. اتصال این سرورها با دستور تک‌خطی `claude mcp add` انجام می‌شود.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: پروتکل MCP (Model Context Protocol) چه وظیفه‌ای دارد؟

الف) مدل‌های جدید هوش مصنوعی را آموزش می‌دهد.

ب) یک استاندارد باز و یکپارچه فراهم می‌سازد تا ایجنت بتواند با دیتابیس‌ها، سرویس‌های ابری، مرورگرها و ابزارهای خارجی ارتباط برقرار کند.

ج) کدهای جاوااسکریپت را به تایپ‌اسکریپت کامپایل می‌کند.

د) رمز عبور سیستم را نگه‌داری می‌نماید.

✅ **پاسخ صحیح: گزینه «ب»**
پروتکل MCP پلی جهان‌شمول است که به ایجنت‌ها اجازه می‌دهد به جای محدود شدن به ترمینال محلی، با تمام سرویس‌های دنیای وب سخن بگویند.

📚 اصطلاحات تخصصی این درس:

MCP

Model Context Protocol: an open standard for plugging external tool servers into any agent harness.

Tool

A named capability the harness lets the model invoke, such as reading a file, running a shell command or fetching a page.

Agent

A model wired to tools and a loop: it reads, acts, checks the result, and goes again until the task is done or you stop it.

🧠 ماژول VIBE-PATTERNS-4

فصل چهارم: الگوها و تکنیک‌های وایب کدینگ

پرامپت‌ها، الگوها و هارمونی‌هایی که کارایی شما را چند برابر می‌کنند؛ مرز میان یک توسعه‌دهنده مسلط وایب‌کدینگ با یک فرد مبتدی در همین تکنیک‌ها و شیوه هدایت ایجنت نهفته است.

درس 19 | کارگاه عملی 🛠️ | ⏱️ زمان مطالعه: ۹ دقیقه

۱۹. الگوی مشخصات → تست‌ها → کد (Spec → Tests → Code)

ID: cc-18-spec-tests-code

خلاصه اجرایی: وقتی مشخصات کار ابتدا به صورت تست‌های ناموفق (قرمز) نوشته شوند، ایجنت یک هدف قابل اندازه‌گیری دارد و تا سبز شدن تست‌ها متوقف نمی‌شود.

اگر ابتدا کد را بنویسید و سپس بگویید برایش تست بنویس، تست‌ها فقط «کدی که نوشته شده را توصیف می‌کنند» نه رفتاری که واقعاً انتظار داشتید! رویکرد وایب کدینگ استاندارد: ابتدا مشخصات کار را به زبان ساده بنویسید؛ از ایجنت بخواهید تست‌های رد شونده (Red) را بنویسد؛ سپس کد اصلی را پیاده کند تا همه تست‌ها سبز شوند.

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Describe the spec in prose

.Don't dive into types yet. Write 4-6 sentences that describe the behavior, the inputs, the outputs, the edge cases

راستی‌آزمایی: .The spec is short, behavior-focused, and lists at least 2 edge cases

گام 02: Have the agent write tests first

.Ask explicitly for tests before any implementation

راستی‌آزمایی: .You see a test file and a failing run with N failures, where N matches the assertions

گام 03: Now write the code

.Ask for the implementation. Iterate until green

راستی‌آزمایی: .Tests pass. Code is shaped by the spec, not by the first thing that came to mind

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چرا تست‌هایی که «قبل» از پیاده‌سازی نوشته می‌شوند ارزش بالاتری از تست‌های بعد از پیاده‌سازی دارند؟

الف چون تست‌های اولیه حجم کمتری از رم را می‌گیرند.

ب چون تست‌های قبلی رفتار و نیازمندی واقعی را مشخص می‌کنند؛ در حالی که تست‌های پس از کد صرفاً رفتاری را که کد نوشته‌شده تصادفاً دارد تایید می‌نمایند.

ج چون پایتورچ با تست‌های قبلی سریع‌تر کار می‌کند.

د چون آنتروپیک تست‌های پس از کد را نمی‌پذیرد.

✅ **پاسخ صحیح: گزینه «ب»**

تست‌های پیشین بیانگر قرارداد (Contract) و مشخصات واقعی هستند و از سوگیری مدل در توجیه اشتباهات کد خودش جلوگیری می‌کنند.

🧩 **اصطلاحات تخصصی این درس:**

Spec

A written description of what to build and how you'll know it's right, agreed before the agent starts writing code.

درس 20 | شبیه‌ساز مکالمه | ⏱ زمان مطالعه: ۸ دقیقه

۲۰. بهبود تدریجی و گام‌های اتمیک (Iterative Refinement)

ID: cc-19-iterative-refinement

خلاصه اجرایی: اشتباه رایج: دادن پرامپت‌های عظیم ۵۰۰ کلمه‌ای برای ساخت کل برنامه در یک ضربه! تکنیک درست: تحویل برش‌های کوچک و قابل تست.

به جای این‌که بگویید «کل داشبورد را بساز با ۸ تا کامپوننت و ۳ تا API»، کار را به قطعات کوچک بشکنید: ۱. ابتدا رابط و تایپ داده‌ها و حالت خالی (Empty State) ۲. سپس لودینگ و حالت دریافت خطا (Error State) ۳. سپس واکشی داده واقعی ۴. در نهایت قابلیت فیلتر و جستجو. هر مرحله کوتاه است، کانتکست خلوت می‌ماند و خطاها فوراً کشف می‌شوند.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Add the dashboard. Start with just the loading skeleton — no real data yet.

کلود: Smallest slice first.

کاربر: Good. Now wire the /api/stats endpoint and replace the skeleton with real numbers. Loading + error states only — no empty state yet.

کلود: Adding fetch + states.

کاربر: Now add the empty state for new users.

سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چرا کوچک نگه داشتن ابعاد تسک‌ها، کارایی و اعتمادپذیری ایجنت را افزایش می‌دهد؟

الف) چون تسک کوچک در توکن‌های کمتری اجرا شده، بازخورد آنی دارد و خطاها بلافاصله قبل از انباشته شدن کشف و حل می‌شوند.

ب) چون مدل‌های هوش مصنوعی حافظه بیشتر از ۱۰۰ خط را نمی‌فهمند.

ج) چون هزینه اینترنت کمتر می‌شود.

د) چون گیت نمی‌تواند فایل‌های بزرگ را کامیت کند.

پاسخ صحیح: گزینه «ب»

گام‌های اتمیک و تدریجی (Iterative) از سردرگمی مدل جلوگیری کرده و امکان اعتبارسنجی دقیق در هر مرحله را فراهم می‌سازند.

درس 21 — مفهوم بنیادین — زمان مطالعه: ۷ دقیقه

۲۱. چه زمانی مداخله کنیم و چه زمانی بگذاریم اجرا شود؟ (When to Interrupt)

ID: cc-20-when-to-interrupt

خلاصه اجرایی: غریزه اولیه انسان برای مداخله در همان ثانیه اولی که چیزی را اشتباه می‌بیند معمولاً غلط است. بگذارید چرخه ایجنت نتیجه را مشاهده کند.

زمان درست مداخله: - وقتی ایجنت به سمت دستورات خطرناک یا نامربوط پیش می‌رود. - وقتی مسیر را کاملاً اشتباه متوجه شده است. زمان نادرست مداخله: - وقتی اروری در کنسول ظاهر شده اما ایجنت هنوز در حال خواندن ارور برای اصلاح آن است! بگذارید خودش خروجی ارور را بخواند و خطای خود را تصحیح کند؛ مداخله بیجا در میانه مشاهده، تمرکز مدل را به هم می‌ریزد.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: کدام مورد دلیل «موجه و درستی» برای مداخله در میانه چرخه ایجنت است؟

- **الف** مشاهده اولین خطای تایپی در ترمینال.
- **ب** مشاهده ارور قرمز کامپایلر در لاگ‌ها در حالی که ایجنت در حال خواندن آن است.
- **ج** **وقتی ایجنت به سمت دستورات خطرناک، حذف اشتباهی فایل‌ها یا مسیری کاملاً خلاف هدف حرکت می‌کند.**
- **د** وقتی سرعت تولید توکن‌ها کاهش یافته است.

✅ **پاسخ صحیح: گزینه «ج»**

تنها زمانی مداخله کنید که اقدام خطرناکی در شرف وقوع باشد یا ایجنت هدف را به کلی گم کرده باشد؛ در سایر موارد بگذارید ایجنت خروجی را مشاهده و خودش رفع کند.

درس 22 | شبیه‌ساز مکالمه | ⏱ زمان مطالعه: ۷ دقیقه

۲۲. کدریویو با کلود (Code Review with Claude)

ID: cc-21-code-review

خلاصه اجرایی: بازبینی خودکار با ایجنت، نقاط کوری را که چشم انسان بعد از ساعت‌ها کدنویسی دیگر نمی‌بیند برملا می‌کند.

بهترین فرمول پرامپت برای ریویو: «تغییرات این git diff را مثل یک مهندس نرم‌افزار ارشد که از باگ‌های پروداکشن، نشتی حافظه و پیچیدگی‌های بی‌مورد متنفر است بررسی کن؛ فقط روی باگ‌های پنهان، ایرادات امنیتی و موارد لبه‌ای تمرکز کن».

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Review the staged diff. Senior posture, hostile to churn. Be specific.

کلود: Reading the staged diff.

کلود: Three issues: 1. orderRouter accepts `userId` from the body and trusts it — should come from the session, not user input. 2. The new test only covers happy path. Missing: invalid items array, negative quantity, unauthenticated request. 3. Schema migration adds `total_cents INT NOT NULL` with no default — will fail in prod if any row exists. Add a backfill or default.

کاربر: Fix #1 now. I'll add tests for #2 myself.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: بهترین فرمول و لحن برای درخواست کدریویو (Code Review) از ایجنت چیست؟

- **الف** آیا کد من قشنگ است؟
- **ب** **این تغییرات را مانند یک مهندس ارشد نقاد که از باگ‌های پروداکشن، نشتی حافظه و پیچیدگی‌های بی‌جا متنفر است بازبینی کن.**
- **ج** کدها را تایید کن تا کامیت کنم.
- **د** همه چیز را به زبان پایتون بازنویسی کن.

✅ **پاسخ صحیح: گزینه «ب»**

دادن نقش یک مهندس ارشد سخت‌گیر مانع از تملق‌گویی مدل شده و توجه آن را مستقیماً به باگ‌های لبه‌ای و پرمخاطره جلب می‌نماید.

🧵 **اصطلاحات تخصصی این درس:**

Human review
A person reading the agent's work before it ships. The only step that judges whether the change is right, not just whether it passes.

درس 23 | مفهوم بنیادین 💡 | ⏱️ زمان مطالعه: ۸ دقیقه

۲۳. جریان کار جفت‌برنامه‌نویسی (Pair-Programming Flow)

ID: cc-22-pair-programming-flow

خلاصه اجرایی: در جفت‌برنامه‌نویسی کلاسیک، یک نفر تایپ می‌کند (راننده) و یک نفر فکر می‌کند (ناوبری‌کننده). با کلود کد، شما همیشه در جایگاه ناوبری‌کننده هستید.

شما جهت را تعیین می‌کنید، معماری را مشخص می‌سازید، کیفیت خروجی را می‌سنجید و تصمیم‌گیری می‌کنید. کلود کارهای خسته‌کننده، تایپ کردن‌ها، سرچ‌ها و چک کردن ایمپورت‌ها را بر عهده می‌گیرد. این هم‌افزایی، سرعت توسعه را به طرز خیره‌کننده‌ای دگرگون می‌کند.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: در یک رابطه جفت‌برنامه‌نویسی با ایجنت هوش مصنوعی، نقش اصلی شما چیست؟

الف تایپ سریع کدهایی که ایجنت دیکته می‌کند.

ب نقش معمار و ناوبری‌کننده (Navigator): تعیین اهداف، جهت‌دهی معماری، وضع قیود و سنجش کیفیت نتایج.

ج تماشاگر منفعل بدون حق دخالت.

د نوشتن تست‌های دستی بدون استفاده از ترمینال.

✅ **پاسخ صحیح: گزینه «ب»**

توسعه‌دهنده به عنوان هدایت‌کننده استراتژیک عمل می‌کند و ایجنت به عنوان راننده اجرایی سریع و ماهر.

🧵 اصطلاحات تخصصی این درس:

Agent

A model wired to tools and a loop: it reads, acts, checks the result, .and goes again until the task is done or you stop it

🏆 ماژول REAL-PROJECTS-5

فصل پنجم: پروژه‌های دنیای واقعی و استقرار

مطالعات موردی از صفر تا صد در مقیاس واقعی، نه مثال‌های اسباب‌بازی و تکراری: ساخت فیچرهای مدرن با Next.js، کدنویسی تست‌محور (TDD)، مهاجرت کدهای قدیمی و میرا، انتشار ایمن در پروداکشن، و تحویل پروژه نهایی.

درس 24 | پروژه عملی 🚀 | ⏱ زمان مطالعه: ۱۰ دقیقه

۲۴. ساخت یک فیچر کامل در Next.js (Building a Next.js Feature)

ID: cc-23-nextjs-feature

خلاصه اجرایی: پیاده‌سازی صفر تا صد یک ویژگی مدرن: خواندن ساختار پوشه‌ها، تعریف روت و صفحه، ساخت کامپوننت، اتصال استیت و تست کامل.

یک مطالعه موردی واقعی در Next.js App Router: ایجنت ابتدا درخت فایل‌های پروژه را اسکن می‌کند تا با معماری پروژه (مثلاً وجود مسیرهای داینامیک، نحوه تفکیک Server/Client Components و ابزارهای استایل‌دهی) همگام شود. سپس صفحه و روت مربوطه را پیاده‌سازی کرده و تست اجرای صحیح صفحه را انجام می‌دهد.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Add /pricing as a new route with a server-component page. Pull copy from a `pricing.json` next to the page. Add a test that asserts the page renders the headlines.

کلود: Plan: create the route folder, the JSON, the page, the test.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چرا در ساخت فیچر Next.js، پرامپت همزمان مسیر (Route)، ساختار داده و تست را در یک دستور تعیین می‌کند؟

الف برای این‌که پرامپت طولانی‌تر به نظر برسد.

ب چون دادن تمام اضلاع یک ویژگی در کنار هم، قیود کاملی به ایجنت می‌دهد تا معماری را منسجم و بدون فرضیات اشتباه بسازد.

ج چون فریم‌ورک Next.js دستورات چندبخشی را قبول نمی‌کند.

د چون در غیر این صورت ایجنت متوقف می‌شود.

✅ **پاسخ صحیح: گزینه «ب»**

تعریف همزمان ورودی، خروجی و معیار پذیرش (تست) باعث همگرایی فوری مدل در اولین تلاش می‌شود.

درس 25 | پروژه عملی 🚀 | ⏱ زمان مطالعه: ۹ دقیقه

۲۵. رویکرد تست‌محور (Writing Tests-First in Action)

ID: cc-24-tests-first

خلاصه اجرایی: سریع‌ترین و مطمئن‌ترین حلقه TDD که تا به حال تجربه کرده‌اید: شما مشخصات را می‌گویید؛ ایجنت تست ناموفق می‌سازد و سپس تا پاس شدن آن کد می‌زند.

مشاهده عملی چرخه ساخت ماژول محاسباتی: ابتدا تست برای مقادیر مرزی، اعداد منفی و حالت‌های نامعتبر نوشته شده و شکست می‌خورد؛ سپس پیاده‌سازی منطق بیزینس انجام شده و با فرمان تست مجدد، سبز شدن تمام سناریوها به ثبت می‌رسد.

بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Build a `RateLimiter` class. New limiter accepts (maxRequests, windowMs). Each call to .check(key) returns true if under limit, false if over. Window slides. Memory only.

کلود: Tests first. Writing failing tests for: under limit, exactly at limit, over limit, window slides, distinct keys are independent.

کلود: All red. Now writing the implementation.

کلود: Green. Want me to refactor for clarity, or ship as-is?

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: دیدن شکست هر ۵ تست قبل از پیاده‌سازی کدها چه چیزی را اثبات می‌کند؟

الف اثبات می‌کند که کامپیوتر خراب است.

ب **اثبات می‌کند که تست‌ها توخالی نیستند و شرایط مد نظر ما واقعاً هنوز پیاده‌سازی نشده است؛ پس وقتی بعداً سبز شوند، نشانه موفقیت قطعی است.**

ج نشان می‌دهد که باید فریم‌ورک تست را عوض کنیم.

د هیچ ارزشی ندارد و وقت را تلف می‌کند.

✅ **پاسخ صحیح: گزینه «ب»**

اثبات حساسیت تست‌ها به فقدان ویژگی، پایه و اساس اطمینان در مهندسی تست‌محور است.

🧩 **اصطلاحات تخصصی این درس:**

Agent

A model wired to tools and a loop: it reads, acts, checks the result, and goes again until the task is done or you stop it.

درس 26 | پروژه عملی 🚀 | ⏱ زمان مطالعه: ۹ دقیقه

۲۶. مهاجرت کدهای قدیمی (Migrating Legacy Code)

ID: cc-25-legacy-migration

خلاصه اجرایی: مهاجرت‌های بزرگ معمولاً شکست می‌خورند اگر تیم بخواهد همه‌چیز را در یک تغییر ناگهانی دگرگون کند. کلود کد این کار را با ایمنی و لایه‌به‌لایه جلو می‌برد.

نمونه ارتقای ماژول‌های کدهای قدیمی (مثلاً تبدیل کدهای جاوااسکریپت قدیمی به تایپ‌اسکریپت تایپ‌شده سخت‌گیرانه): تبدیل مرحله‌به‌مرحله توابع، اضافه کردن فایل‌های d.ts، اطمینان از سازگاری کدهای بالادستی با تست‌های رگرسیون، و تثبیت کامیت‌ها پس از هر گام موفق.

💬 بازپخش نشست شبیه‌سازی‌شده (Sandbox Transcript Replay) — Claude Code Agentic Loop

کاربر: Migrate src/utils/format.js to TypeScript. Don't touch any other file. Existing tests must still pass.

کلود: Reading the file and its tests.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: قید کلیدی که مهاجرت کدهای قدیمی به روش برگ‌به‌برگ (Leaf-by-leaf) را ایمن می‌سازد چیست؟

- **الف** حذف تمام متغیرهای سراسری.
- **ب** **تغییر ندادن اینترفیس عمومی توابع و ثابت نگه‌داشتن قرارداد ورودی-خروجی تا کدهای بالادستی دچار شکست نشوند.**
- **ج** تبدیل همزمان تمام فایل‌های ریپازیتوری در یک کامیت.
- **د** خاموش کردن تایپ‌چکر.

✅ **پاسخ صحیح: گزینه «ب»**

با حفظ ثبات API عمومی، قطعات داخلی ماژول‌ها بدون ایجاد اثر پروانه‌ای یا شکستن سایر بخش‌های پروژه به‌روزرسانی می‌شوند.

درس 27 | کارگاه عملی 🛠️ | ⏱️ زمان مطالعه: ۸ دقیقه

۲۷. استقرار و ارسال به پروداکشن (Shipping to Production)

ID: cc-26-shipping-to-production

> **خلاصه اجرایی:** استقرار در محیط زنده و عملیاتی، حساس‌ترین بخش چرخه است. ایجنت در این مرحله نقش ناظر سفت‌وسخت و مجری چک‌لیست را ایفا می‌کند.

قبل از این‌که کدی وارد پروداکشن شود، از ایجنت بخواهید چک‌لیست چهارگانه را اجرا کند: ۱. اجرای تست‌ها و تایید ۱۰۰٪ ۲. بررسی کامل نوع‌ها با TypeCheck ۳. بررسی خطاهای نحوی با Linter ۴. ساخت پکیج بیلد نهایی بدون اخطار. همچنین تدوین برنامه بازگشت (Rollback Plan) پیش از دپلوی توسط ایجنت آماده می‌شود.

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Run the pre-deploy gauntlet

.Tests, types, lint, build — all four green before anything touches prod

راستی‌آزمایی: .Either all four pass, or you get a precise failure to triage

گام 02: Inspect the migration

Have the agent read the migration that's about to run, summarize what it changes, and flag any non-reversible .operations

راستی‌آزمایی: You get a 1-paragraph summary plus an explicit list of irreversible ops (DROPs, NOT NULL on backfilled .columns, etc.)

گام 03: Draft the rollback plan

.Before deploying, have the agent write the rollback steps you'll run if something breaks

راستی‌آزمایی: .A markdown file exists with the rollback steps before you deploy. Future you will thank present you

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: چرا برنامه بازگشت (Rollback Plan) باید «قبل» از استقرار نوشته شود نه بعد از بروز فاجعه؟

الف) چون بعد از خرابی در پروداکشن، استرس بالاست و تصمیم‌گیری درست دشوار می‌شود؛ مراحل رول‌بک باید با ذهن آرام تست و آماده باشد.

ب) چون گیت بعد از دپلوی اجازه پول‌بک نمی‌دهد.

ج) چون سرورهای کلود به فایل بازگشت نیاز دارند تا روشن شوند.

د) چون این یک قانون اداری بی‌اهمیت است.

✅ **پاسخ صحیح: گزینه «ب»**

در هنگام حادثه در پروداکشن زمان برای فکر کردن نیست؛ چک‌لیست دقیق بازگشت باید از قبل تست و تایید شده باشد.

درس 28 | پروژه نهایی 🏆 | زمان مطالعه: ۱۵ دقیقه ⏱️

۲۸. پروژه پایانی: ساخت و انتشار یک سایدپروژه (Capstone: Ship a Side Project)

ID: cc-27-capstone

خلاصه اجرایی: اثبات نهایی تسلط شما بر وایب کدینگ: انتخاب یک ایده واقعی و کوچک، معماری با ایجنت، ساخت در چند چرخه کوتاه و استقرار نهایی در اینترنت.

پروژه نهایی نباید یک کپی عظیم از توییتر باشد؛ یک ابزار کوچک اما واقعی انتخاب کنید که واقعاً کار کند: مثلاً یک خلاصه‌ساز متون، یک سازمان‌دهنده بوک‌مارک‌ها یا یک ماشین حساب مالی اختصاصی. با آموخته‌های ۲۷ درس گذشته، ساختار را پایه‌ریزی کنید، با گام‌های مشخصات→تست→کد پیش بروید و محصول را در یک پلتفرم مستقر کنید!

🛠️ مراحل عملی و گام‌به‌گام (Hands-on Walkthrough):

گام 01: Pick the idea — small and real

Not a clone of Twitter. Not 'a CRM for X.' Pick one tiny tool with one user — maybe yourself. A bookmark organizer. A meeting-notes summarizer. A tip calculator with split logic. The smaller, the better.

راستی‌آزمایی: Your idea fits in three sentences. If it doesn't, scope down.

گام 02: Set up the repo with the agent

Use Claude Code to scaffold. Pick a stack you mostly know — capstone is not the time to learn three new frameworks.

راستی‌آزمایی: You have a runnable app on localhost:3000.

گام 03: Build the spine in 3–4 small turns

Use what you learned: spec → tests → code, small slices, mark-complete on each. Each turn ends in a runnable, testable state.

راستی‌آزمایی: Each turn produces a slice you could ship. If a turn produces something not-shippable, scope it smaller next time.

گام 04: Deploy

Vercel, Netlify, Cloudflare Pages, Railway — pick whichever has a one-command deploy.

راستی‌آزمایی: There is a public URL. You can text it to a friend.

گام 05: Write a 1-paragraph postmortem

The capstone isn't done until you've reflected. What did the agent do better than you expected? Where did it lose? What prompts would you reuse next time?

راستی‌آزمایی: POSTMORTEM.md exists, is 1-2 paragraphs, and lists at least one thing you'd do differently. You're done with the course.

📝 سنجش درک مطلب و ارزیابی (Quiz)

پرسش 1: کدام یک انتخاب «نامناسبی» برای سایدپروژه نهایی (Capstone) است؟

الف یک ابزار کوچک و واقعی که نیاز روزمره خودتان را برطرف کند.

ب تلاش برای شبیه‌سازی یک پلتفرم عظیم تجاری مانند یک CRM کامل برای بیمارستان‌ها یا کلون توییتر در اولین تجربه.

ج یک خلاصه‌ساز لینک‌های خبری.

د یک افزونه ساده برای مرتب‌سازی فایل‌ها.

✅ **پاسخ صحیح: گزینه «ب»**

سایدپروژه باید کوچک، متمرکز و به اتمام‌رسیدنی باشد؛ ایده‌های عظیم و نامشخص در دام پیچیدگی‌ها می‌افتند و هدف یادگیری چرخه کامل را مخدوش می‌کنند.

🛠️ بخش ضمیمه شماره ۱

چیت‌شیت جامع و مرجع سریع کلود کد (Claude Code Cheat Sheet)

تمام فرامین ترمینال، دستورات اسلش، کلیدهای میانبر، سطوح دسترسی، تنظیمات، هوک‌ها، سرورهای MCP و شیوه‌نامه‌های طلایی وایب کدینگ در یک نگاه.

۱. نصب و شروع سریع (Install & Start)

نصب کلاینت Claude Code روی رایانه محلی و شروع اولین نشست کاری در پوشه پروژه.

```
curl -fsSL https://claude.ai/install.sh | bash
```

نصب بومی در مک و لینوکس (توصیه‌شده، بدون نیاز به Node.js)

```
irm https://claude.ai/install.ps1 | iex
```

نصب بومی در سیستم‌عامل ویندوز با PowerShell

```
npm install -g @anthropic-ai/claude-code
```

نصب سراسری از طریق پکیج منیجر npm

```
claude
```

شروع نشست تعاملی با ایجنت در پوشه فعال فعلی

```
claude "توضیح ساختار پروژه و نقطه ورود"
```

اجرای مستقیم یک پرامپت یک‌خطی بدون معطلی

```
claude --version
```

نمایش نسخه نصب‌شده و بررسی سلامت دستور در PATH سیستم

```
claude doctor
```

بررسی اتصال شبکه، کلید API، وضعیت لاگین و سلامت محیط

۲. دستورات اسلش پرکاربرد (Slash Commands)

با فشردن کاراکتر / در ترمینال، دستورات توکار فراخوانی می‌شوند؛ دستورات اختصاصی در مسیر `.claude/commands/` تعریف می‌گردند.

`/init`

اسکن پروژه و ساخت فایل اولیه CLAUDE.md شامل معماری، دستورات تست و ساخت

`/clear`

پاک‌سازی کامل نشست و خالی کردن کانتکست (حافظه CLAUDE.md حفظ می‌شود)

`/compact`

خلاصه‌سازی گفتگوی فعلی برای آزادسازی فضای پنجره کانتکست با حفظ پیوستگی کار

`/resume`

بازیابی و ادامه آخرین نشست کاری بسته شده در پروژه

`/rewind`

ماشین زمان: بازگرداندن فایل‌ها یا مکالمه به چک‌پوینت‌های قبلی بدون نیاز به دستکاری گیت

`/model`

سوئیچ آنی میان مدل‌های Claude (مثلاً Opus برای معماری، Haiku برای ادیت سریع)

`/cost`

نمایش دقیق توکن‌های مصرفی ورودی/خروجی، توکن‌های کش‌شده و هزینه دلاری نشست

`/doctor`

عیب‌یابی پیکربندی، اتصالات و پرمیشن‌های محیط کاری جاری

۳. کلیدهای میانبر و کنترل ورودی (Keyboard & Input)

کلیدهایی که مرز میان یک ناظر منفعل با یک راهبر مسلط را در محیط ترمینال مشخص می‌کنند.

Shift + Tab

چرخش میان مودهای امنیتی: Normal (پرسش قبل از ادیت) → Auto-Accept → Plan Mode

Ctrl + C (یکبار)

متوقف کردن فوری عملیات در حال اجرای ایجنت بدون خروج از برنامه

Ctrl + C (دوبار)

خروج کامل از محیط خط فرمان کلود کد

Escape (دوبار)

باز کردن منوی بازگردانی سریع چک‌پوینت‌ها (/rewind) برای بازگشت به عقب

@filename

ارجاع مستقیم به یک فایل خاص و افزودن آن به کانتکست پرامپت (مثلاً `@src/auth.ts چرا اکسپایر میشه؟`)

!command

اجرای مستقیم یک دستور شل خارج از کنترل ایجنت (مثلاً `!npm test`)

#rule

افزودن یک دستورالعمل سریع به حافظه موقت سشن جاری

۴. مودها و سطوح دسترسی (Modes & Permissions)

تعیین این‌که ایجنت بدون اجازه شما تا چه اندازه اجازه تغییرات دارد.

Normal (Ask)

حالت پیش‌فرض: فایل‌ها را آزادانه می‌خواند؛ اما قبل از هر ادیت یا دستور Bash تاییدیه می‌گیرد.

--permission-mode plan

مود برنامه‌ریزی: فقط خواندن و تحلیل؛ هیچ ویرایش فایلی مجاز نیست تا نقشه راه تایید شود.

--dangerously-skip-permissions

رد شدن از تمام تاییدیه‌ها؛ ایجنت بدون توقف فایل‌ها را ویرایش و دستورات را اجرا می‌کند (فقط در کانتینر یا داکر ایزوله)

--allowedTools "Read" "Bash(npm test)"

اجازه صریح به ابزارهای ایمن خاص بدون پرسیدن تاییدیه مکرر

۵. حافظه مانا و فایل‌های تنظیمات (Memory & Config)

محل استقرار دستورالعمل‌ها، تنظیمات و قوانین پروژه؛ این فایل‌ها باید در مخزن گیت کامیت شوند.

CLAUDE.md

دستورالعمل‌های سطح پروژه؛ قرار گرفته در ریشه پروژه که در ابتدای هر نشست بارگذاری می‌شود.

~/.claude/CLAUDE.md

تنظیمات و دستورالعمل‌های شخصی شما در تمام پروژه‌ها (قالب پاسخ، سبک کدنویسی فردی).

.claude/commands/*.md

محل نگه‌داری اسلش‌کامندهای اختصاصی و تیمی به صورت فایل‌های مارک‌داون.

.claude/config.json

فایل پیکربندی اختصاصی تنظیمات پیشرفته ابزار و کلیدهای پیکربندی.

۶. هوک‌های خودکار (Hooks: Pre & Post Tool)

دستورات قطعی سیستمی که به محض رخداد وقایع در حلقه ایجنت به صورت خودکار اجرا می‌شوند.

`PreToolUse`

اجرا پیش از فراخوانی ابزار. اگر کد خروج اسکریپت ۲ باشد، فراخوانی مسدود و دلیل آن به ایجنت گزارش می‌شود.

`PostToolUse`

اجرا بلافاصله پس از اتمام ابزار (مثلاً اجرای خودکار Prettier بعد از هر ویرایش فایل).

۷. سرورهای پروتکل متنی مدل (MCP Servers)

اتصال ایجنت به ابزارهای جانبی: پایگاه‌های داده، مرورگرها، مستندات، ایشیوها و سرویس‌های ابری.

`claude mcp add github -- npx -y @modelcontextprotocol/server-github`

اتصال سرور گیت‌هاب برای مدیریت ایشیوها، کامیت‌ها و پول‌ریکوئست‌ها

`claude mcp add postgres -- npx -y @modelcontextprotocol/server-postgres`

اتصال مستقیم ایجنت به دیتابیس Postgres برای خواندن اسکیما و کوئری

`claude mcp list`

نمایش لیست تمامی سرورهای MCP متصل‌شده به نشست جاری

`claude mcp remove`

حذف و قطع ارتباط با یک سرور MCP مشخص

۸. ساب‌ایجنت‌ها و مهارت‌ها (Subagents & Skills)

تفویض وظایف سنگین پژوهشی یا تخصص‌های ویژه به کانتکست‌های مجزا برای حفظ چابکی ایجنت اصلی.

`Agent Tool`

تولید یک ساب‌ایجنت مجزا با کانتکست تمیز که پس از اتمام کار، خلاصه‌ای از یافته‌ها را برمی‌گرداند.

`SKILL.md`

پکیج استاندارد مهارت شامل توضیحات و دستورالعمل‌های تخصصی در مسیر `.claude/skills//`

۹. اسکریپت‌نویسی و استفاده بدون رابط کاربری (Headless CLI)

اجرای فرامین کلود کد در خطوط لوله CI/CD، اسکریپت‌های بش و کرون‌جاب‌ها بدون نیاز به رابط تعاملی.

```
claude -p "پرامپت شما"
```

اجرای تک‌پرامپت، دریافت پاسخ و خروج فوری (مناسب برای پایپ‌لاین‌ها)

```
claude -p "..." --output-format json
```

خروجی گرفتن به صورت آبجکت ساختاریافته استاندارد JSON

```
claude -p "..." --max-turns 10
```

محدود کردن حداکثر چرخه‌های رفت‌وبرگشت ایجنت برای جلوگیری از حلقه‌های بی‌پایان

۱۰. کار همزمان موازی با گیت ورک‌تری (Git Worktrees)

اجرای همزمان چندین ایجنت روی یک مخزن واحد در شاخه‌های مختلف بدون تداخل فایل‌ها.

```
git worktree add ../my-app-auth -b feat/auth
```

ایجاد یک چک‌اوت مجزا از پروژه روی شاخه جدید feat/auth

```
cd ../my-app-auth && claude
```

اجرای ایجنت مستقل روی تسک جدید در دایرکتوری موازی بدون دستکاری شاخه اصلی

```
git worktree remove ../my-app-auth
```

پاک‌سازی ورک‌تری پس از مرج شاخه در گیت

۱۱. مدیریت هزینه‌ها و محدودیت توکن‌ها (Cost & Token Limits)

کجا توکن‌ها مصرف می‌شوند و چگونه نشست‌ها را بهینه‌سازی و اقتصادی نگه داریم؟

```
Prompt Caching
```

آنتروپیک پرامپت‌ها و فایل‌های قبلی را کش می‌کند؛ تا ۹۰٪ تخفیف در هزینه‌های ورودی کانتکست‌های تکراری.

```
/compact اجرای منظم
```

خلاصه کردن مکالمات قدیمی مانع از انفجار توکن‌های کانتکست در سشن‌های طولانی می‌شود.

```
/model تفکیک مدل‌ها با
```

برای نوشتن کامپوننت‌های ساده یا تست‌های جزیی از Haiku استفاده کنید؛ سرمایه Opus را برای معماری و ریفکتورهای پیچیده نگه دارید.

۱۲. قواعد و شیوه‌نامه‌های طلایی وایب کدینگ (Golden Best Practices)

عادات هوشمندانه‌ای که کیفیت خروجی هر مدلی را چندین برابر بهتر می‌کنند.

برنامه‌ریزی قبل از ویرایش

هر تغییری که بیش از یک فایل را دستخوش دگرگونی می‌کند را ابتدا در مود Plan آغاز کنید.

مشخصات قبل از تست، تست قبل از کد

چرخه Spec → Tests → Code خطاناپذیرترین قطب‌نمای ایجنت است.

پرامپت‌های کوتاه و متمرکز

پروژه‌ها را لایه به لایه بسازید؛ پرامپت‌های عظیم باعث سردرگمی مدل و هدررفت توکن می‌شوند.

اعتماد به حلقه در دیباگ

وقتی اروری رخ می‌دهد، سریعاً تایپ نکنید؛ بگذارید ایجنت خروجی ارور را بخواند و خودش را اصلاح کند.

۱۳. پرسش‌های متداول (Claude Code FAQ)

پاسخ‌های شفاف به سوالات پرتکرار برنامه‌نویسان در شروع کار با کلود کد.

آیا کلود کد جایگزین ادیتور من (VS Code/Cursor) می‌شود؟

خیر! کلود کد در کنار ادیتور شما درون ترمینال باز است؛ شما فایل‌ها را در ادیتور می‌بینید و ایجنت در ترمینال تغییرات را اعمال می‌کند.

آیا کدهای پروژه من برای آموزش مدل استفاده می‌شوند؟

خیر! بر اساس سیاست‌های رسمی تجاری Anthropic، کدهای ارسالی از طریق API و Claude Code هرگز برای آموزش مدل‌ها استفاده نمی‌شوند.

تفاوت اصلی با چت‌بات‌های وب در چیست؟

چت‌بات وب هیچ دسترسی به فایل‌ها یا دستورات شما ندارد و باید کپی-پیست کنید؛ کلود کد ابزارهای واقعی ترمینال، گیت و تست را در دست دارد.

📖 بخش ضمیمه شماره ۲

واژه‌نامه و اصطلاح‌نامه تخصصی هوش مصنوعی و وایب کدینگ

فرهنگ تشریحی ۸۳ اصطلاح کلیدی مهندسی مدل‌های زبانی بزرگ، یادگیری عمیق، و توسعه نرم‌افزار ایجنتیک با برگردان روان فارسی.

ایجنت هوشمند

Agent

سیستمی که در یک چرخه تکرار (While Loop) با اتکا به مدل زبانی، تصمیم می‌گیرد چه ابزاری را فراخوانی کند، آن را اجرا کرده، خروجی را بررسی می‌کند و تا اتمام تسک به کار خود ادامه می‌دهد.

مکانیزم توجه (Attention)

Attention

مکانیزمی در شبکه‌های ترنسفورمر که در آن هر توکن، مجموع وزن‌داری از مقادیر سایر توکن‌ها را بر اساس میزان ارتباط و شباهت ضرب داخلی (Dot Product) میان بردار پرس‌وجو (Query) و کلید (Key) محاسبه می‌کند.

هم‌راستاسازی (Alignment)

Alignment

چالش فنی و اخلاقی در هوش مصنوعی برای تطبیق رفتار مدل با اهداف، ارزش‌ها، ترجیحات و نیت‌های انسانی، به ویژه در شرایط لبه‌ای پیش‌بینی‌نشده.

مدل خودبازگشتی (Autoregressive)

Autoregressive

مدلی که توکن بعدی را بر اساس تمامی توکن‌های پیشین پیش‌بینی کرده و سپس آن خروجی را به عنوان ورودی گام بعد به خود تزریق می‌کند. مدل‌های GPT، LLaMA و Claude همگی خودبازگشتی هستند.

تابع فعال‌ساز

Activation Function

تابعی که پس از هر لایه خطی اعمال می‌شود تا خاصیت غیرخطی را به شبکه تزریق کند. بدون آن، انباشتن لایه‌های خطی به یک تبدیل خطی واحد تبدیل می‌شود. متداول‌ترین‌ها: ReLU, GELU, SiLU.

بهینه‌ساز ادم (Adam)

Adam (Optimizer)

الگوریتم تخمین ممان تطبیقی (Adaptive Moment Estimation) که ممان اول (Momentum) را با نرخ یادگیری تطبیقی برای هر پارامتر (ممان دوم) ترکیب می‌کند و بدون نیاز به تنظیمات پیچیده روی اکثر تسک‌ها عالی کار می‌کند.

بهینه‌ساز AdamW

AdamW

نگارش اصلاح‌شده Adam با تجزیه وزنی غیروابسته (Decoupled Weight Decay). در ادم سنتی منظم‌سازی L2 تحت تاثیر نرخ یادگیری تغییر می‌کرد اما AdamW تجزیه وزن را مستقیماً روی وزن‌ها اعمال می‌کند.

سیستم گرادیان خودکار

Autograd

موتور محاسباتی در پایتورچ و فریم‌ورک‌های یادگیری عمیق که عملیات روی تنسورها را در یک گراف محاسباتی ثبت کرده و در پس‌انتشار (Backward Pass) مشتقات را به صورت خودکار محاسبه می‌کند.

اندازه دسته (Batch Size)

Batch Size

تعداد نمونه‌های آموزشی که در یک گذر رفت‌وبرگشت (Forward/Backward Pass) پردازش می‌شوند. دسته بزرگ‌تر پایداری گرادیان را افزایش داده اما حافظه VRAM بیشتری می‌طلبد.

پس‌انتشار خطا (Backpropagation)

Backpropagation

الگوریتم بنیادین آموزش شبکه‌های عصبی بر پایه قاعده زنجیره‌ای دیفرانسیل که نشان می‌دهد هر وزن به چه میزانی در خطای نهایی سهم داشته است.

پنجره زمینه / طول کانتکست (Context Window)

Context Window

حداکثر تعداد توکن‌هایی که یک مدل زبانی می‌تواند در یک لحظه به صورت همزمان دریافت، درک و پردازش کند (شامل پرامپت، تاریخچه چت، فایل‌ها و پاسخ تولیدی).

زنجیره تفکر (CoT)

Chain of Thought (CoT)

تکنیکی در پرامپت‌نویسی که مدل را وادار می‌کند مسائل پیچیده را گام‌به‌گام و مرحله‌ای استدلال کند تا احتمال خطای منطقی کاهش یابد.

شبکه عصبی پیچشی (CNN)

CNN (Convolutional Neural Network)

معماری پیشگام یادگیری عمیق در پردازش تصویر که با اعمال فیلترهای لغزان (Kernels) ویژگی‌های محلی مانند لبه‌ها و بافت‌ها را استخراج می‌کند.

کودا (CUDA)

CUDA

پلتفرم محاسبات موازی و رابط برنامه‌نویسی انویدیا که به نرم‌افزارها اجازه می‌دهد از قدرت پردازشی فوق‌العاده تراشه‌های گرافیکی (GPU) برای محاسبات هوش مصنوعی بهره ببرند.

قطعه‌بندی اسناد (Chunking)

Chunking

فرآیند خرد کردن اسناد و متون طولانی به تکه‌های کوچک‌تر و معنادار پیش از تولید بردار و ذخیره‌سازی در پایگاه‌داده‌های برداری در سیستم‌های RAG.

یادگیری تقابلی (Contrastive Learning)

Contrastive Learning

روشی از یادگیری که در آن به مدل آموزش داده می‌شود نمونه‌های مشابه را در فضای برداری به یکدیگر نزدیک و نمونه‌های نامرتبط را از هم دور نگه دارد.

شباهت کسینوسی (Cosine Similarity)

Cosine Similarity

معیاری ریاضی برای سنجش شباهت جهت دو بردار در فضای چندبعدی بدون توجه به اندازه بردار؛ اصلی‌ترین متریک در جستجوی معنایی و پایگاه‌داده‌های برداری.

انتروپی متقاطع (Cross-Entropy)

Cross-Entropy

تابع خطای استاندارد برای مسائل دسته‌بندی و پیش‌بینی توکن در مدل‌های زبانی که اختلاف میان توزیع احتمال پیش‌بینی‌شده و برچسب واقعی را می‌سنجد.

افزودن و غنی‌سازی داده‌ها (Data Augmentation)

Data Augmentation

تکنیک‌هایی برای تولید داده‌های جدید از روی داده‌های موجود (مثل چرخش تصویر یا جایگزینی کلمات هم‌معنی) برای بهبود تعمیم مدل و کاهش بیش‌برازش.

بخش رمزگشا (Decoder)

Decoder

بخشی از ترنسفورمر که توکن‌ها را به ترتیب و خودبازگشتی بر مبنای بردارهای زمینه و توکن‌های قبلی تولید می‌کند.

مدل‌های انتشاری (Diffusion)

Diffusion Model

کلاسی از مدل‌های مولد (مثل Stable Diffusion و Midjourney) که ابتدا نویز تدریجی به داده اضافه کرده و سپس با یادگیری فرآیند حذف نویز، تصاویر یا ویدیوهای واقع‌گرایانه خلق می‌کنند.

بهینه‌سازی مستقیم ترجیحات (DPO)

DPO (Direct Preference Optimization)

رویکرد مدرن در هم‌راستاسازی LLMها که بدون نیاز به آموزش یک مدل پاداش جداگانه (برخلاف RLHF سنتی)، مدل را مستقیماً روی جفت‌های مورد ترجیح انسان بهینه‌سازی می‌کند.

دراپ‌اوت (Dropout)

Dropout

روش منظم‌سازی در شبکه‌های عصبی که در جریان آموزش، تعدادی از نورون‌ها را به صورت تصادفی خاموش می‌کند تا شبکه به ویژگی‌های تصادفی وابسته نشود.

مقدار ویژه (Eigenvalue)

Eigenvalue

عددی در جبر خطی که نشان می‌دهد یک تبدیل خطی، بردار ویژه مربوطه را چقدر کشیده یا فشرده می‌کند؛ کلیدی در تحلیل مولفه‌های اصلی (PCA).

جاسازی / بردارسازی (Embedding)

Embedding

تبدیل کلمات، متون، یا داده‌ها به یک بردار عددی فشرده در فضای چندبعدی پیوسته که روابط معنایی را با فواصل ریاضی حفظ می‌کند.

بخش رمزگذار (Encoder)

Encoder

بخشی از ترنسفورمر که کل متن ورودی را به طور همزمان خوانده و آن را به مجموعه‌ای از بردارهای بازنمایی متنی غنی تبدیل می‌کند.

دوره آموزشی (Epoch)

Epoch

یک بار عبور کامل تمامی نمونه‌های مجموعه داده آموزشی از درون الگوریتم یادگیری شبکه عصبی.

ویژگی (Feature)

Feature

یک ویژگی قابل اندازه‌گیری یا متغیر مستقل از پدیده مورد مشاهده که به عنوان ورودی به مدل‌های هوش مصنوعی ارائه می‌شود.

یادگیری با چند نمونه (Few-Shot)

Few-Shot

رویکردی در پرامپت‌نویسی که در آن چند نمونه از جفت‌های ورودی-خروجی به مدل داده می‌شود تا فرمت و سبک پاسخ‌دهی را قبل از اجرای تسک اصلی یاد بگیرد.

تیونینگ و تنظیم دقیق (Fine-tuning)

Fine-tuning

فرآیند بازآموزی یک مدل پیش‌آموزش‌دیده روی یک دیتاست خاص و تخصصی با نرخ یادگیری پایین به منظور انطباق آن با دامنه‌ای ویژه.

فراخوانی توابع / ابزارها (Function Calling)

Function Calling

قابلیت یک مدل زبانی برای تشخیص نیاز به ابزار خارجی و تولید یک خروجی ساختاریافته (مثل JSON) شامل نام تابع و آرگومان‌های لازم جهت اجرا در سرور.

نرده‌های ایمنی / گاردریل (Guardrails)

Guardrails

لایه‌های اعتبارسنجی و قوانین امنیتی پیش و پس از اجرای مدل زبانی برای اطمینان از عدم نقض قوانین، ایمنی داده‌ها و پاسخگویی مسئولانه.

ترنسفورمر پیش‌آموزش‌دیده مولد (GPT)

GPT

معماری خانواده ترنسفورمر تنها-دیکودر (Decoder-only) توسعه‌یافته توسط OpenAI برای تولید زبان طبیعی با مقیاس وسیع.

شبکه مولد رقابتی (GAN)

GAN (Generative Adversarial Network)

معماری شامل دو شبکه رقیب: مولد (Generator) که داده‌های جعلی می‌سازد و متمایزکننده (Discriminator) که داده واقعی و جعلی را تفکیک می‌کند.

گرادیان (Gradient)

Gradient

برداری از مشتقات جزئی که جهت و نرخ تندترین شیب صعود تابع خطا را نسبت به پارامترهای مدل مشخص می‌کند.

گرادیان نزولی (Gradient Descent)

Gradient Descent

الگوریتم پایه‌ای بهینه‌سازی که پارامترها را در خلاف جهت بردار گرادیان حرکت می‌دهد تا مقدار تابع خطا به حداقل برسد.

ابَرپارامتر (Hyperparameter)

Hyperparameter

تنظیماتی که قبل از شروع آموزش مدل توسط مهندس مشخص می‌شوند و توسط خود مدل یاد گرفته نمی‌شوند (مثل نرخ یادگیری، اندازه دسته و تعداد لایه‌ها).

توهم هوش مصنوعی (Hallucination)

Hallucination

پدیده‌ای که در آن مدل زبانی اطلاعاتی کاملاً نادرست، جعلی یا ساختگی را با اعتمادبه‌نفس بالا و به عنوان حقیقت بیان می‌کند.

استنتاج / اجرا (Inference)

Inference

مرحله استفاده عملیاتی از مدل آموزش‌دیده برای پاسخگویی به درخواست‌های جدید بدون به‌روزرسانی وزن‌ها.

سوگیری استقرایی (Inductive Bias)

Inductive Bias

مجموعه مفروضاتی که در معماری یک الگوریتم تعبیه شده تا برای داده‌های نادیده اولویت‌های مشخصی قائل شود (مثلاً ترجمه ناپذیری فضایی در CNN).

کتابخانه JAX

JAX

فریم‌ورک محاسبات عددی گوگل بر پایه کامپایلر XLA که امکان دیفرانسیل‌گیری خودکار، برداری‌سازی خودکار و اجرای شتاب‌یافته روی TPU/GPU را فراهم می‌آورد.

حافظه پنهان کلید-مقدار (KV Cache)

KV Cache

تکنیک بهینه‌سازی در استنتاج ترنسفورمرها که با ذخیره بردارهای Key و Value توکن‌های گذشته در رم، از محاسبه مجدد آن‌ها در توکن‌های بعدی جلوگیری کرده و سرعت را چند برابر می‌کند.

فضای نهان (Latent Space)

Latent Space

فضایی فشرده و کم‌بعد که در آن داده‌های پیچیده در قالب ویژگی‌های معنایی انتزاعی نمایش داده می‌شوند.

نرخ یادگیری (Learning Rate)

Learning Rate

اندازه گامی که بهینه‌ساز در جهت منفی گرادیان برمی‌دارد؛ بزرگ بودن آن باعث ناپایداری و کوچک بودن آن باعث کندی شدید همگرایی می‌شود.

مدل زبانی بزرگ (LLM)

LLM (Large Language Model)

مدل‌های هوش مصنوعی با ده‌ها تا صدها میلیارد پارامتر که با تحلیل حجم‌های عظیم از متون اینترنت، توانایی درک، بازنویسی و تولید زبان و کد را به دست آورده‌اند.

انطباق کم‌رتبه (LoRA)

LoRA (Low-Rank Adaptation)

تکنیک کارآمد فاین‌تیونینگ (PEFT) که با تثبیت وزن‌های اصلی مدل و تزریق دو ماتریس کوچک کم‌رتبه، حجم حافظه مورد نیاز برای آموزش تخصصی را تا ۹۰٪ کاهش می‌دهد.

تابع زیان / تابع هزینه (Loss Function)

Loss Function

فرمول ریاضی که میزان فاصله و خطای خروجی فعلی مدل را با نتیجه ایده‌آل و مورد انتظار اندازه‌گیری می‌کند.

محاسبات با دقت ترکیبی (Mixed Precision)

Mixed Precision

استفاده همزمان از فرمت‌های محاسباتی ۱۶ بیتی (FP16/BF16) و ۳۲ بیتی (FP32) در جریان آموزش برای صرفه‌جویی در حافظه VRAM و دوبرابر کردن سرعت آموزش.

معماری ترکیبی متخصصان (MoE)

MoE (Mixture of Experts)

معماری شبکه‌ای که در آن چندین زیرشبکه تخصصی (Experts) وجود دارد و یک شبکه مسیریاب (Router) برای هر توکن تنها ۱ یا ۲ متخصص فعال را انتخاب می‌کند تا بدون افزایش هزینه محاسباتی، ظرفیت مدل بالا برود.

پروتکل متنی مدل (MCP)

MCP (Model Context Protocol)

استاندارد باز معرفی‌شده توسط آنتروپیک برای اتصال امن، استاندارد و یکپارچه مدل‌های هوش مصنوعی و ایجنت‌ها به ابزارها، فایل‌ها، سرورها و پایگاه‌های داده.

خطای عددی NaN

NaN (Not a Number)

نمادی برای مقادیر نامعتبر ریاضی (مانند تقسیم بر صفر)؛ در آموزش شبکه‌های عصبی معمولاً نشانه انفجار گرادیان است.

نرمال‌سازی (Normalization)

Normalization

فرآیند مقیاس‌بندی ورودی‌ها یا فعال‌سازها (مثل LayerNorm یا BatchNorm) تا میانگین و واریانس در طول لایه‌های عمیق پایدار بمانند.

بیش‌برازش (Overfitting)

Overfitting

وضعیتی که مدل جزئیات و نویزهای داده‌های آموزشی را حفظ کرده و کارایی خود را روی داده‌های جدید و نادیده از دست می‌دهد.

بهینه‌ساز (Optimizer)

Optimizer

الگوریتمی که فرمول به‌روزرسانی وزن‌ها را بر اساس گرادیان‌ها مدیریت می‌کند (مانند SGD, AdamW).

پارامتر / وزن‌های شبکه (Parameter)

Parameter

متغیرهای درونی شبکه عصبی (وزن‌ها و بایاس‌ها) که در جریان آموزش به‌روز شده و دانش مدل را درون خود حمل می‌کنند.

شاخص سردرگمی (Perplexity)

Perplexity

معیاری برای سنجش کیفیت مدل زبانی؛ عکس احتمال پیش‌بینی توکن‌های واقعی، به طوری که مقدار کمتر نشان‌دهنده پیش‌بینی دقیق‌تر و اطمینان بالاتر است.

دقت و بازخوانی (Precision & Recall)

Precision & Recall

دو معیار اصلی سنجش مدل‌های دسته‌بندی؛ دقت نشان‌دهنده درستی پیش‌بینی‌های مثبت و بازخوانی نشان‌دهنده پوشش تمامی نمونه‌های مثبت واقعی است.

مهندسی پرامپت (Prompt Engineering)

Prompt Engineering

دانش و هنر نگارش دستورالعمل‌ها، زمینه‌ها و مثال‌ها برای هدایت مدل زبانی به سوی خروجی دقیق، ساختاریافته و باکیفیت.

تزریق پرامپت (Prompt Injection)

Prompt Injection

یک آسیب‌پذیری امنیتی که در آن ورودی‌های غیرقابل‌اعتماد کاربر یا وب، دستورات سیستمی مدل را دور زده و رفتار ایجنت را تغییر می‌دهند.

کیولوآرا (QLoRA)

QLoRA

تکنیک فوق‌پیشرفته فاین‌تیونینگ که مدل اصلی را با کوانتیزاسیون ۴ بیتی فشرده کرده و امکان آموزش مدل‌های بسیار بزرگ را روی کارت‌های گرافیک ارزان خانگی فراهم می‌آورد.

تولید مبتنی بر بازیابی (RAG)

RAG (Retrieval-Augmented Generation)

معماری هوشمندی که پیش از پاسخگویی مدل زبانی، مرتبط‌ترین اسناد و اطلاعات را از پایگاه‌های داده بازیابی کرده و به عنوان زمینه به پرامپت مدل تزریق می‌کند.

یادگیری تقویتی از بازخورد انسانی (RLHF)

RLHF (Reinforcement Learning from Human Feedback)

فرآیند هم‌راستاسازی که در آن یک مدل پاداش بر اساس رتبه‌بندی‌های انسانی ساخته شده و مدل اصلی با الگوریتم PPO برای کسب پاداش بالاتر بهینه‌سازی می‌شود.

کوانتیزاسیون / فشرده‌سازی عددی (Quantization)

Quantization

تبدیل وزن‌ها و فعال‌سازهای مدل از اعداد اعشاری دقیق (مثل FP32 یا FP16) به فرمت‌های کم‌بیت‌تر (مثل INT8 یا INT4) برای کاهش شدید حجم حافظه و افزایش سرعت اجرا.

تابع ReLU

ReLU

تابع فعال‌ساز یکسوساز خطی که مقادیر منفی را صفر کرده و مقادیر مثبت را بدون تغییر عبور می‌دهد؛ سریع‌ترین و متداول‌ترین تابع فعال‌ساز.

معیار ارزیابی ROUGE

ROUGE

مجموعه معیارهایی برای ارزیابی خودکار سیستم‌های خلاصه‌سازی و ترجمه با مقایسه هم‌پوشانی n-gram ها با متون مرجع انسانی.

جستجوی معنایی (Semantic Search)

Semantic Search

جستجو بر اساس معنی و مفهوم عبارات به جای تطابق کلمه به کلمه کلیدواژه‌ها، با استفاده از فواصل برداری در امبدینگ‌ها.

پخش جریانی توکن‌ها (Streaming)

Streaming

ارسال تدریجی توکن‌های تولیدی مدل به محض محاسبه شدن به سمت کلاینت بدون نیاز به انتظار برای اتمام کل متن.

توجه به خود (Self-Attention)

Self-Attention

مکانیزم هسته ترنسفورمر که به هر توکن در یک جمله اجازه می‌دهد مستقیماً با تمامی توکن‌های دیگر همان جمله ارتباط برقرار کند و روابط گرامری و معنایی را دریابد.

فاین‌تیونینگ نظارت‌شده (SFT)

SFT (Supervised Fine-Tuning)

آموزش مدل روی دیتاست‌های پرسش و پاسخ یا دستورالعمل با خروجی‌های ایده‌آل دست‌چین‌شده انسانی.

تابع سافت‌مکس (Softmax)

Softmax

تابعی ریاضی که برداری از اعداد حقیقی را به توزیعی از احتمالات تبدیل می‌کند که مجموع آن‌ها دقیقاً برابر با ۱ است.

معماری دسته جمعی ایجنت‌ها (Swarm)

Swarm

الگویی که در آن مجموعه‌ای از ایجنت‌های تخصصی و مستقل، با پیام‌رسانی و تقسیم کار بدون کنترل مرکزی بر روی یک هدف مشترک فعالیت می‌کنند.

پرامپت سیستمی (System Prompt)

System Prompt

دستورالعمل‌های بنیادین و اولیه‌ای که پیش از پیام‌های کاربر تعریف می‌شوند و شخصیت، نقش، قوانین و نحوه پاسخگویی مدل را تعیین می‌کنند.

تنسور (Tensor)

Tensor

آرایه‌ای چندبعدی از اعداد که ساختار پایه‌ای داده‌ها و محاسبات در یادگیری عمیق و شبکه‌های عصبی را تشکیل می‌دهد.

توکن (Token)

Token

کوچک‌ترین واحد متنی که مدل زبانی پردازش می‌کند؛ می‌تواند یک کاراکتر، بخشی از کلمه، یا یک کلمه کامل باشد (در انگلیسی تقریباً هر کلمه ۱.۳ توکن است).

دما / تصادفی بودن (Temperature)

Temperature

پارامتری برای کنترل درجه تصادفی بودن و خلاقیت پاسخ‌های مدل؛ دمای صفر پاسخ‌های کاملاً قطعی و دمای بالا پاسخ‌های متنوع و غیرقابل‌پیش‌بینی تولید می‌کند.

یادگیری انتقالی (Transfer Learning)

Transfer Learning

روشی که در آن دانش کسب‌شده از یک تسک جامع (مثل خواندن اینترنت) به یک تسک خاص و جدید منتقل می‌شود.

ترنسفورمر (Transformer)

Transformer

معماری انقلابی شبکه عصبی معرفی‌شده در سال ۲۰۱۷ که با اتکا به مکانیزم Attention، ستون فقرات تمامی مدل‌های مدرن هوش مصنوعی شد.

کم‌برازش (Underfitting)

Underfitting

وضعیتی که مدل آن‌قدر ساده یا آموزش ناکافی است که حتی قادر به یادگیری الگوهای داده‌های آموزشی هم نیست.

خودرمزگذار متغیر (VAE)

VAE (Variational Autoencoder)

مدل مولدی که داده‌ها را به یک توزیع احتمالاتی پیوسته در فضای نهان فشرده کرده و امکان تولید نمونه‌های جدید و پیوسته را می‌دهد.

پایگاه داده برداری (Vector Database)

Vector Database

دیتابیس تخصصی برای ذخیره، ایندکس و جستجوی فوق‌سریع میان میلیون‌ها بردار با الگوریتم‌های همسایگی مانند HNSW.

وزن شبکه عصبی (Weight)

Weight

ضریب‌های عددی درون لایه‌های شبکه که میزان اثرگذاری یک سیگنال ورودی بر خروجی نورون بعدی را تعیین می‌کنند.

تجزیه وزن (Weight Decay)

Weight Decay

روشی برای منظم‌سازی شبکه که در جریان آموزش، جریمه‌ای متناسب با بزرگی وزن‌ها به تابع خطا اضافه می‌کند تا از رشد بی‌رویه وزن‌ها جلوگیری شود.

یادگیری بدون نمونه (Zero-Shot)

Zero-Shot

توانایی مدل زبانی برای حل یک مسئله بدون نیاز به ارائه هیچ‌گونه نمونه اولیه، صرفاً بر اساس توصیف تسک در پرامپت.



پایان کتابچه و آغاز سفر وایب کدینگ شما

شما اکنون تمامی مبانی، ابزارها، الگوهای فکری، و جریان‌های کاری وایب کدینگ با Claude Code را آموخته‌اید. به یاد داشته باشید که در این پارادایم نوین، قدرت شما در توانایی شما برای تعیین شفاف اهداف، معماری تمیز، و هدایت ایجنت خلاصه می‌شود.

> 🌟 **پیشنهاد بعدی:** همین حالا ترمینال خود را باز کنید، دستور `claude` را در یکی از پروژه‌هایتان تایپ کنید و اولین سایدپروژه واقعی خود را با ریتم Spec → Tests → Code خلق کنید!

تهیه‌شده با عشق برای جامعه توسعه‌دهندگان مدرن وب · پاییز ۱۴۰۳ (مطابق با ۲۰۲۶)